

Align·ROI·Calibration·좌표계

Glass Align에서 Image-Machine Coordinate Transform까지

-
- | | |
|----|---|
| 01 | 8일차. 3개 Align Mark로 370×470 Glass 중심 정렬하기 |
| 02 | 12일차. ROI와 Align 기반 Coordinate Transform |
| 03 | 13일차. Alignment 심화: Matching에서 X/Y/Theta까지 |
| 04 | 14일차 A. Calibration: Image Coordinate를 Machine Coordinate로 변환하기 |
| 05 | 14일차 B. Camera Calibration: Intrinsic · Extrinsic · Distortion |
| 06 | 15일차. Coordinate System: Image · Camera · Stage · Machine · Product |
-

원본 20개 Markdown의 내용·코드·표·수식을 보존하여 주제별로 편집

8일차. 3개 Align Mark로 370×470 Glass 중심 정렬하기

특별 학습 주제 · 예상 학습 시간: 6~8시간 · 난이도: 중급 · 중요도: ● 반드시 알아야 함
이번 문서는 요청에 따라 코드 구현 없이 개념, 수식, Teaching 및 자동 Align Logic에 집중한다.

1. 오늘 해결할 문제

Glass 크기:

FORMULA · CALCULATION

X 방향 Width = 370 mm

Y 방향 Height = 470 mm

Glass 설계 중심 = (0, 0)

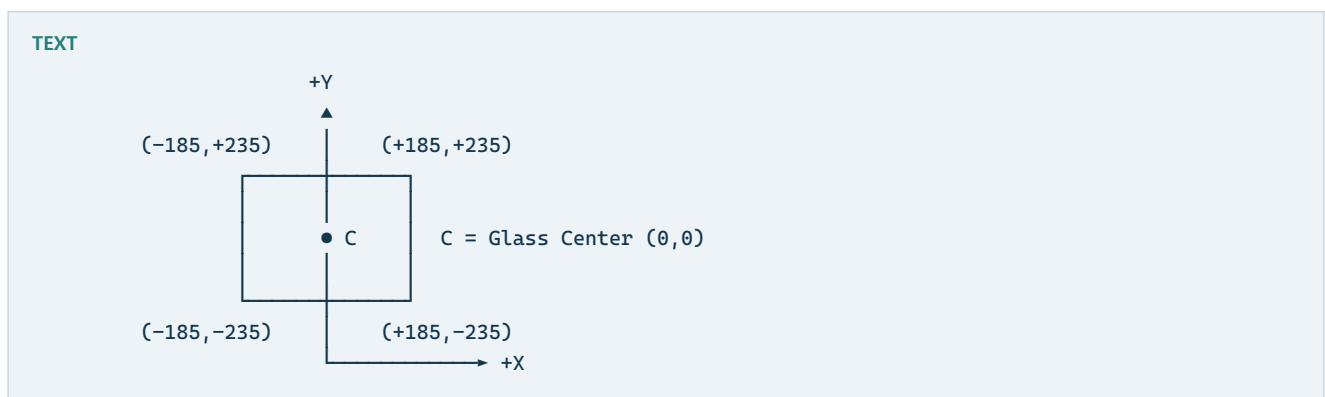
Glass 위에 Align Mark가 3개 있고 Camera 한 대로 각 Mark를 확인한다고 가정한다. 목표는 다음과 같다.

1. 처음 기준 Glass를 올바른 자세로 놓고 Align 기준을 등록한다.
2. 이후 Glass가 X/Y로 이동하거나 회전되어 투입되어도 Mark 3개를 자동 검출한다.
3. 검출한 Mark로 현재 Glass 중심좌표와 회전각 **X/Y/Theta** 를 계산한다.
4. Stage를 보정하거나 검사 ROI를 Transform하여 기준 Glass와 같은 좌표계로 검사한다.

2. 가장 먼저 정할 좌표계

2.1 Glass 설계 좌표계

Glass 중심을 원점으로 둔다.

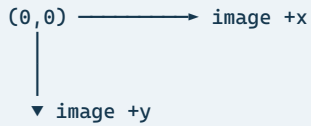


- X 범위: **-185~+185 mm**
- Y 범위: **-235~+235 mm**
- Mark의 설계 위치는 이 중심 기준 좌표로 저장한다.

2.2 Image 좌표계

보통 Camera Image는 좌상단 원점이고 아래쪽이 +y다.

TEXT



Glass/Machine 좌표의 +Y가 위쪽이라면 Image y와 부호가 반대다. Image Pixel 좌표로 바로 각도를 계산하면 회전 부호를 잘못 해석할 수 있으므로 먼저 Calibration을 적용해 공통 Machine 좌표계로 변환한다.

2.3 Machine/Stage 좌표계

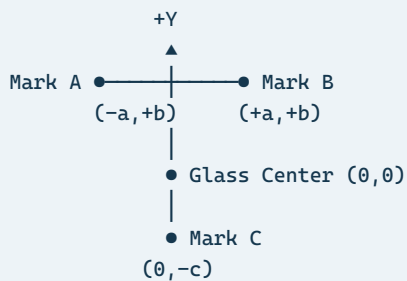
실제 Stage가 사용하는 X/Y/Theta 좌표계다. 다음을 명확히 정의해야 한다.

- X/Y축의 +방향
- 길이 단위(mm 또는 μm)
- Theta의 +방향(CCW인지 CW인지)
- Stage 회전 중심(Pivot)
- Camera가 고정인지 이동하는지
- Glass가 이동하는지 Camera가 이동하는지

3. 3개 Mark 배치 예제

질문의 배치를 아래와 같은 대표적인 삼각형 Layout으로 가정한다.

TEXT



- Mark A/B: Y축을 기준으로 좌우 대칭
- Mark C: Y축 위에 위치
- 세 Mark가 Glass 중심을 둘러싸는 넓은 삼각형을 구성

370×470 mm Glass의 설명용 예:

FORMULA · CALCULATION

Mark A = (-150, +200) mm

Mark B = (+150, +200) mm

Mark C = (0, -200) mm

Glass Center = (0, 0) mm

이 Mark 위치는 설명용이다. 실제 값은 Glass Drawing의 Mark 중심 좌표를 사용한다.

[!NOTE]

만약 질문에서 말한 "X축 대칭 Mark 2개"가 위·아래 대칭을 뜻한다면 X와 Y 역할만 바꾸면 된다. 뒤의 Y축 방향 Pair 공식을 사용한다.

4. 일반적인 Vision Align의 목적

Align은 현재 Glass 자세와 기준 Glass 자세의 차이를 구하는 과정이다.

FORMULA · CALCULATION

기준 자세: X_{ref} , Y_{ref} , Θ_{ref}

현재 자세: X_{cur} , Y_{cur} , Θ_{cur}

Pose Error:

$\Delta X = X_{cur} - X_{ref}$

$\Delta Y = Y_{cur} - Y_{ref}$

$\Delta \Theta = \Theta_{cur} - \Theta_{ref}$

보통 Align은 다음 둘 중 하나에 사용한다.

4.1 Physical Alignment

Stage를 X/Y/Theta로 움직여 Glass 자체를 기준 자세에 맞춘다.

TEXT

Mark 검출 → Pose 계산 → Stage 보정 → 재촬영 → 허용 오차 확인

4.2 Virtual Alignment

Glass를 움직이지 않고 검사 ROI와 검사 좌표를 현재 Glass 자세에 맞게 이동·회전한다.

TEXT

Mark 검출 → Pose 계산 → ROI Transform → 현재 위치에서 검사

장비에 정밀 X/Y/Theta Stage가 있으면 Physical Align을, Cycle Time이나 기구 조건상 움직이기 어렵다면 Virtual Align을 사용한다. 두 방식을 결합할 수도 있다.

5. 사용자가 생각한 방법은 맞는가?

생각한 흐름은 기본적으로 맞다.

TEXT

```

기준 Glass 투입
↓
Camera로 Mark 중심 검출
↓
Glass를 이동/회전하여 기준 자세로 맞춤
↓
Align 기준 등록(Teaching)
↓
다음 Glass부터 Mark 자동 검출
↓
기준 대비 X/Y/Theta 계산
↓
Stage 또는 ROI 자동 보정
  
```

다만 정렬된 Image만 저장하는 것으로는 부족하다. 다음 데이터가 함께 필요하다.

- Mark A/B/C 각각의 Template 또는 검출 조건
- Mark ID와 순서
- 각 Mark의 Glass 중심 기준 설계 좌표 (x_i, y_i)
- 기준 등록 시 각 Mark의 Machine 좌표
- Camera Pixel → Machine 좌표 Calibration
- Mark마다 Camera가 이동해야 하는 Stage 촬영 위치
- Glass 기준 중심과 Stage 회전 중심의 관계
- Match Score, Search Range, 허용 위치/각도 오차
- Recipe와 Calibration Version

즉, 등록 대상은 단순 Align Image가 아니라 **Mark Geometry**를 포함한 **Align Recipe**다.

5.1 핵심 질문: 처음에 Align Mark Image가 반드시 있어야 하는가?

정확한 답은 사용하는 Mark 검출 방식에 따라 다르다.

TEXT

```

실제 Template Image가 항상 필수인가? → 아니오
Mark에 대한 사전 정의가 필요한가? → 예
  
```

Camera가 Image를 촬영했다고 해서 SW가 아무 정보 없이 “이것이 Align Mark다”라고 자동으로 판단할 수는 없다. 시스템에는 최소한 다음 중 하나가 있어야 한다.

1. 실제로 촬영해 등록한 Mark Template Image
2. CAD 또는 Drawing에서 가져온 Mark 형상 Model
3. 원, 십자, 사각형처럼 수학적으로 정의된 Geometry
4. 밝기, 크기, 면적, 원형도 등 Blob 조건
5. QR/AprilTag처럼 미리 정의된 Fiducial 규칙과 ID

즉 **Mark Image**가 무조건 필요한 것은 아니지만, 어떤 형상을 **Mark**라고 부를 것인지에 대한 기준은 무조건 필요하다.

5.2 Mark 종류별 Template Image 필요 여부

Align Mark 형태	일반적인 검출 방법	실제 Template Image
검은 원/흰 원	Threshold → Blob/Contour → 원 Fit	보통 불필요
Ring Mark	Edge → Circle/Ellipse Fit	보통 불필요
단순 십자 +	Edge/Line 검출 → 두 선 교점	보통 불필요
사각형/마름모	Contour/Corner 검출	보통 불필요
고유 Logo/복잡한 Pattern	Template/Pattern Matching	필요 또는 강력 권장
QR/AprilTag 계열	규격 Decoder/Corner 검출	별도 촬영 Template 불필요
주변 Pattern과 매우 비슷한 Mark	Pattern Matching + 예상 ROI	필요 권장

예: 검은 원형 Align Mark



이 경우에는 Mark Template Image 없이도 **검은 원** , **예상 지름** , **면적 범위** , **원형도** , **예상 ROI** 가 Mark의 기준이 된다.

예: 십자 Align Mark**FORMULA · CALCULATION**

Image 촬영
 ↓
 두 방향 Edge/Line 검출
 ↓
 수평 계열 중심선 + 수직 계열 중심선 계산
 ↓
 두 중심선의 교점 = Align Mark 중심

Glass가 회전되어 십자가 기울어져 있어도 두 Line의 각도 범위를 넓게 검색하고 교점을 계산할 수 있다. 이 경우 "수평선은 항상 0°"라고 고정하면 안 된다.

예: 복잡한 고유 Pattern

Mark가 Logo나 복잡한 Pattern이면 기준 Glass에서 촬영한 Image를 Template로 등록하고, Runtime에 위치와 각도를 함께 검색하는 Pattern Matching이 일반적이다.

5.3 Image가 전혀 없는 최초 설치 시에는 어떻게 시작하는가?

자동 Align을 처음 개발하거나 장비를 Setup할 때는 아직 촬영 Template가 없을 수 있다. 이때 다음 순서로 **최초 Teaching Image**를 만든다.

TEXT

Glass Drawing/CAD에서 Mark 설계좌표 확인
 ↓
 기준 Glass를 Jig 또는 대략적인 Nominal 위치에 투입
 ↓
 Stage를 첫 번째 Mark의 예상 위치로 이동
 ↓
 넓은 Search ROI 또는 전체 FOV 촬영
 ↓
 Geometry Finder 또는 작업자 확인으로 Mark 후보 선택
 ↓
 Exposure/Focus/조명을 조정해 좋은 Mark Image 획득
 ↓
 필요하면 이 Image를 Template로 등록
 ↓
 Mark 중심 정의와 Glass 중심 기준 좌표를 Recipe에 저장

여기서 Drawing의 Mark 좌표와 장비의 Nominal Loading Position이 첫 촬영 위치를 제공한다. 위치 오차가 Camera FOV 보다 크다면 다음 중 하나가 추가로 필요하다.

- 더 넓은 FOV의 Coarse Camera
- Stage Raster Search/Spiral Search
- Glass Edge 또는 Corner를 먼저 찾는 Coarse Align
- 기구 Stopper/Jig/Presence Sensor
- 작업자가 최초 Mark 위치를 지정하는 Teaching UI

[!IMPORTANT]

Mark가 Camera FOV 밖에 있는데 사전 위치 정보도 없다면 알고리즘은 그 Mark를 찾을 수 없다. "자동 검색"은 정해진 Search Range 안에서 수행되는 것이지 장비 전체 공간에서 무한히 찾는 기능이 아니다.

5.4 Template 방식의 최초 등록

Pattern Matching을 사용한다면 최초 한 번은 다음 과정이 필요하다.

1. 기준 Glass를 Nominal 위치에 놓는다.
2. Camera로 Mark가 포함된 Image를 촬영한다.
3. 사용자가 Mark 영역을 ROI로 선택하거나 Geometry Finder가 후보를 제안한다.
4. Mark 중심의 정의를 확인한다.
5. Template Image와 중심 Offset을 저장한다.
6. 허용 Scale/Angle/Search Range와 Match Score를 설정한다.
7. Mark의 Glass 중심 기준 설계좌표를 함께 저장한다.

여기서 Template ROI 중심과 실제 Mark 중심이 반드시 같지는 않다. Template 안에서 Mark 중심이 **(cx,cy)** 어디에 있는지 Center Offset을 저장해야 한다.

5.5 Glass가 들어지면 Mark도 들어지는데 어떻게 중심을 찾는가?

Glass가 X/Y로 이동하고 **Theta** 만큼 회전하면 Mark도 Glass 중심을 기준으로 함께 이동·회전한다.

FORMULA · CALCULATION

현재 Mark 위치 = $R(\text{Theta}) \times \text{Mark 설계좌표} + \text{Glass 중심 Translation}$

하지만 **Mark 자체의 중심은 회전해도 중심 그대로다.**

- 원형 Mark: 원을 회전해도 Center는 동일
- 십자 Mark: 두 선이 회전하지만 교점은 Center
- 사각 Mark: 네 Corner가 회전하지만 대각선 교점/Contour 중심으로 Center 계산
- Template Mark: 회전 Matching이 위치 **(x,y)** 와 각도 **theta** 를 반환

따라서 Runtime Logic은 "Mark가 똑바로 놓여 있을 것"을 가정하지 않고 다음과 같이 검색한다.

TEXT

```

예상 위치 주변 Search ROI 설정
↓
허용 X/Y 이동 범위 검색
↓
허용 Angle 범위 검색 또는 Rotation-invariant Geometry 검출
↓
현재 기울어진 Mark의 중심좌표 검출
↓
세 Mark 중심으로 Glass 전체 X/Y/Theta 계산
  
```


예를 들어 Glass 회전 범위가 $\pm 2^\circ$ 라면 Pattern Matching의 Angle Search를 최소한 그 범위와 여유 Margin까지 설정한다. Geometry 방식에서는 Line/Circle Fit이 해당 각도에서도 안정적으로 중심을 찾는지 검증한다.

5.6 무엇이 기준좌표이고 무엇이 측정좌표인가?

Align에서는 서로 다른 세 좌표를 구분해야 한다.

좌표	의미	예시
Mark 설계좌표 <code>q_i</code>	Glass 중심 <code>(0,0)</code> 에서 Mark가 원래 있어야 하는 위치	$A=(-150, +200)$ mm
Mark Image 좌표	현재 촬영 Image에서 검출한 Mark 중심	$(u,v)=(1023.4, 812.7)$ pixel
Mark Machine 좌표 <code>p_i</code>	Calibration과 촬영 Stage Pose를 적용한 실제 장비 좌표	$(X,Y)=(351.24, 927.51)$ mm

처리 흐름:

TEXT Image에서 Mark Center (u,v) 검출 $\downarrow$ Camera/Stage Calibration Machine Mark Center p_i 계산 $\downarrow$ 3개 p_i와 설계 q_i 비교 Glass Center X/Y와 Theta 계산

Mark의 기준좌표는 Runtime Image에서 새로 만드는 값이 아니라 Drawing/Teaching 단계에 Recipe로 등록되어 있는 값이다. Runtime에서는 현재 Mark 중심만 측정한다.

5.7 Template Matching은 Glass가 회전되어도 찾을 수 있는가?

사용하는 Matcher가 Rotation Search를 지원하고 검색 범위를 올바르게 설정했다면 가능하다.

일반적인 방식:

- Image Pyramid로 Coarse 위치 검색
- 여러 Angle로 Template를 회전하며 비교
- 가장 높은 Score의 X/Y/Theta 선택
- 원본 해상도에서 Sub-pixel Refinement

주의사항:

- 허용 각도가 넓을수록 처리 시간이 증가
- 대칭 Mark는 $90^\circ/180^\circ$ 방향을 구분하지 못할 수 있음
- Mark 일부가 잘리거나 오염되면 Score 저하
- Background에 같은 Pattern이 있으면 오검출 가능
- 예상 ROI와 세 Mark Geometry로 결과를 검증해야 함

원이나 완전 대칭 십자는 자체 각도를 알기 어렵지만 Center 검출은 가능하다. Glass 전체 Theta는 서로 떨어진 Mark 중심을 연결한 Vector로 구하면 된다.

5.8 질문에 대한 최종 답

TEXT

Q. Align Mark Image가 무조건 필요한가?

A. Pattern Matching이면 필요하거나 권장된다.
원/십자/사각 Geometry 검출이면 실제 Template Image 없이도 가능하다.

Q. 아무 기준 없이 Camera가 알아서 Mark를 찾는가?

A. 아니다. Template, CAD/Geometry, 크기·밝기 조건, 예상 ROI 중 최소한 하나 이상의 사전 정의가 필요하다.

Q. Glass가 틀어지면 Mark 중심은 어떻게 찾는가?

A. 허용 이동/회전 범위에서 Pattern 또는 Geometry를 검출해
현재 기울어진 Mark 자체의 중심을 먼저 찾는다.

Q. 그 다음 Align은 어떻게 하는가?

A. 검출 중심을 Machine 좌표로 변환하고, 설계 Mark 좌표와 3점
Rigid Fit하여 Glass 중심 X/Y/Theta를 계산한다.

5.9 가장 권장하는 실제 구성

현재 질문의 370×470 Glass와 3개 Mark에는 다음 구성을 권장한다.

TEXT

[최초 Teaching]

CAD Mark 좌표 + 기준 Glass

- Mark별 좋은 Image 획득
- 단순 원/십자면 Geometry Model 저장
- 복잡한 Mark면 Template Image 저장
- Mark Center Offset과 Glass 중심 기준 좌표 저장

[Runtime]

Nominal Stage Position으로 이동

- 넓은 ROI에서 Mark 검출(X/Y/Angle 허용)
- 현재 Mark Center를 Sub-pixel Refinement
- Machine 좌표로 통합
- 3점 Rigid Fit
- Residual/Pair Distance/Score 검증
- Stage 또는 ROI Align
- 필요 시 재촬영 검증

즉 사용자가 생각한 “기준 Align Mark Image를 등록하고 다음 Glass에서 그 Mark를 찾아 Align”은 올바른 방식 중 하나다. 다만 Mark가 단순 Geometry라면 Image Template 대신 Geometry Model을 사용할 수 있고, 어느 방식이든 Mark 설계 좌표와 Glass 중심의 관계를 반드시 함께 등록해야 한다.

6. Teaching 단계: 최초 기준 등록

Step 1. 기준 Glass의 중심과 방향을 확정

가능한 방법:

- 정밀 Jig/Stopper로 기준 위치 고정
- Glass Edge를 측정해 370×470의 기하 중심 계산
- 외부 계측 기준으로 중심/방향 설정
- 설계 Mark 좌표와 Stage 좌표를 Calibration하여 기준 Pose 계산

Mark를 보면서 사람이 “대충 중앙”으로 맞추는 것만으로는 기준 정확도를 보장하기 어렵다. Teaching Glass가 실제로 어느 위치와 방향에 있어야 하는지 물리적 기준이 필요하다.

Step 2. Mark별 Image 등록

각 Mark에 대해 다음을 등록한다.

- Mark Template 또는 Edge/Blob 검출 조건
- 예상 ROI와 Search ROI
- Polarity: 밝은 Mark인지 어두운 Mark인지
- 중심 정의: Pattern 중심, 원 중심, 교차선 교점 등
- Match Score와 Sub-pixel 적용 여부

세 Mark 모양이 동일하면 잘못 대응될 위험이 있다. 각 Mark의 예상 Stage 위치/ROI를 사용하거나 Mark 디자인을 서로 다르게 만들어 ID를 구분하는 것이 안전하다.

Step 3. Mark 중심을 공통 Machine 좌표로 변환

Camera 한 대가 Mark마다 이동해 촬영한다면 각 Image의 Pixel 좌표는 서로 다른 Camera 위치에서 얻어진다.

FORMULA · CALCULATION

Mark Machine Position
= 촬영 당시 Camera/Stage Pose
+ Image 안에서 검출된 Pixel Offset의 Calibration 변환

보다 일반적으로:

FORMULA · CALCULATION

$$p_i(\text{machine}) = T_{\text{machine} \leftarrow \text{camera}(\text{stage}_i)} \times p_i(\text{image})$$

세 Mark를 같은 Image에서 보지 않는다면 이 단계가 특히 중요하다. 각 Image의 (x, y) 만 서로 빼서 각도를 구하면 잘못된 결과가 된다.

Step 4. Reference Geometry 저장

두 가지 방식이 있다.

방식 A: 설계 좌표 사용 — 권장**FORMULA · CALCULATION**

$$q_A = (-150, +200)$$

$$q_B = (+150, +200)$$

$$q_C = (0, -200)$$

$$\text{Glass Center } q_Center = (0, 0)$$

설계 좌표 **q_i** 와 실제 검출 Machine 좌표 **p_i** 사이 Transform을 계산한다.

방식 B: 잘 정렬된 기준 Glass의 실측 좌표 사용

Teaching 시 Mark Machine 좌표를 Reference로 저장한다.

TEXT

p_A_ref, p_B_ref, p_C_ref

이 방식은 간단하지만 Teaching Glass의 배치 오차가 모든 후속 Glass의 기준에 포함된다. 가능하면 설계 좌표와 정밀 기구 기준을 함께 사용한다.

7. X축 방향 Mark 2개로 회전각 계산하기

Mark A/B가 같은 설계 Y에 있고 좌우로 떨어져 있다고 하자.

FORMULA · CALCULATION

Reference:

$$A = (-a, b)$$

$$B = (+a, b)$$

$$\text{Reference vector } A \rightarrow B = (2a, 0)$$

$$\text{Reference angle} = 0^\circ$$

현재 검출한 Machine 좌표:

FORMULA · CALCULATION

$$A' = (x_A, y_A)$$

$$B' = (x_B, y_B)$$

$$dx = x_B - x_A$$

$$dy = y_B - y_A$$

현재 Glass 회전각:

FORMULA · CALCULATION

$$\text{Theta} = \text{atan2}(dy, dx)$$

기준 Pair가 완전히 수평이 아니라 Reference 각도 **Theta_ref** 가 있다면:

FORMULA · CALCULATION

$$\begin{aligned} \text{DeltaTheta} &= \text{atan2}(dy_{\text{cur}}, dx_{\text{cur}}) \\ &\quad - \text{atan2}(dy_{\text{ref}}, dx_{\text{ref}}) \end{aligned}$$

atan(dy/dx) 대신 **atan2(dy, dx)** 를 사용해야 Quadrant와 **dx=0** 을 올바르게 처리할 수 있다.

Pair 중점과 Glass 중심

A/B 중점은:

FORMULA · CALCULATION

$$M_{AB} = (A' + B') / 2$$

설계상 A/B 중점은 **(0, b)** 이지 Glass 중심 **(0, 0)** 이 아니다. 따라서 중점을 Glass 중심으로 착각하면 Y 방향으로 **b** 만큼 틀린다.

회전행렬을 **R(Theta)** 라 할 때 현재 Glass 중심 **T** 는:

FORMULA · CALCULATION

$$T = M_{AB} - R(\text{Theta}) \times (0,b)$$

즉 Mark Pair의 기준 중심 Offset까지 회전시켜 빼야 한다.

8. Y축 방향 Mark 2개로 각도 계산하기

Mark 두 개가 같은 X에 있고 위·아래로 배치되었다고 하자.

FORMULA · CALCULATION

$$\text{Bottom} = (x_0, -b)$$

$$\text{Top} = (x_0, +b)$$

$$\text{Reference vector Bottom} \rightarrow \text{Top} = (0, 2b)$$

$$\text{Reference angle} = +90^\circ$$

현재 검출 좌표에서:

FORMULA · CALCULATION

$$dx = x_{\text{Top}} - x_{\text{Bottom}}$$

$$dy = y_{\text{Top}} - y_{\text{Bottom}}$$

$$\text{Measured vector angle} = \text{atan2}(dy, dx)$$

$$\text{Glass Theta} = \text{atan2}(dy, dx) - 90^\circ$$

일반식은 항상 같다.

FORMULA · CALCULATION

$$\Delta\text{Theta} = \text{Measured Pair Angle} - \text{Reference Pair Angle}$$

Pair 순서를 뒤집으면 각도가 180° 달라지므로 Bottom/Top 또는 Left/Right ID를 고정해야 한다.

9. 3개 Mark를 모두 사용하는 권장 방법

2개 Mark만으로도 이상적인 2D Rigid Transform의 X/Y/Theta를 계산할 수 있다. 세 번째 Mark는 다음 장점이 있다.

- 검출 오류와 잘못된 Mark 대응 확인
- 한 Mark가 오염되거나 가려진 상황 판별
- 위치 Noise를 평균화
- Mark 간 거리 변화로 Glass/Calibration 이상 검출
- 회전 및 중심 계산 Residual 확인

9.1 Rigid Transform 모델

Glass 중심 기준 설계 Mark 좌표를 q_i , 현재 Machine 좌표를 p_i 라 하면:

FORMULA · CALCULATION

$$p_i = R(\text{Theta}) \times q_i + T$$

여기서:

FORMULA · CALCULATION

$$R(\text{Theta}) = \begin{bmatrix} \cos\text{Theta} & -\sin\text{Theta} \\ \sin\text{Theta} & \cos\text{Theta} \end{bmatrix}$$

$$T = \begin{bmatrix} \text{CenterX} \\ \text{CenterY} \end{bmatrix}$$

q_i 가 Glass 중심 $(0,0)$ 기준이므로 Translation T 가 곧 현재 Glass 중심의 Machine 좌표가 된다.

9.2 3점 최소제곱 Rigid Fit

Mark 3개를 모두 사용하는 일반적인 절차:

1. 설계 Mark 좌표의 중심 q_{bar} 를 구한다.
2. 측정 Mark 좌표의 중심 p_{bar} 를 구한다.
3. 각 Point에서 중심을 뺀 좌표로 최적 회전각을 구한다.
4. $T = p_{\text{bar}} - R(\text{Theta}) \times q_{\text{bar}}$ 로 Translation을 구한다.
5. 각 Mark의 예측 위치와 측정 위치 차이인 Residual을 계산한다.

2D에서는 다음 합으로 회전각을 구할 수 있다.

FORMULA · CALCULATION

$$A = \sum(q'_x \times p'_x + q'_y \times p'_y)$$

$$B = \sum(q'_x \times p'_y - q'_y \times p'_x)$$

$$\text{Theta} = \text{atan2}(B, A)$$

여기서 $q' = q - q_{\text{bar}}$, $p' = p - p_{\text{bar}}$ 다.

이 방법은 특정 Pair 하나만 쓰는 것보다 세 Mark의 측정 Noise를 함께 반영한다.

9.3 Residual 검사

계산된 Pose로 각 Mark 위치를 예측한다.

FORMULA · CALCULATION

$$p_{i_predicted} = R(\text{Theta}) \times q_i + T$$

$$\text{residual}_i = |p_{i_measured} - p_{i_predicted}|$$

Residual이 허용값보다 크면 다음 가능성이 있다.

- Mark 오검출 또는 잘못된 ID
- Camera/Stage Calibration 오류
- Stage Position 또는 Encoder 오류
- Glass Scale/변형/파손
- Mark Drawing 좌표 오류
- Camera Focus/조명 문제

이 경우 역지로 Align을 수행하지 말고 **Align Error** 로 처리한다.

10. 370×470 Glass 수치 예제

설계 Mark:

FORMULA · CALCULATION

$$A = (-150, +200) \text{ mm}$$

$$B = (+150, +200) \text{ mm}$$

$$C = (0, -200) \text{ mm}$$

실제 Glass가 기준보다 다음만큼 이동/회전했다고 가정한다.

FORMULA · CALCULATION

Center Translation T = (+2.000,-1.000) mm

Theta = +0.500° CCW

Camera/Stage Calibration 후 얻은 Mark 좌표가 다음과 같다고 하자.

FORMULA · CALCULATION

$A' \approx (-149.740, +197.683)$ mm

$B' \approx (+150.249, +200.301)$ mm

$C' \approx (+3.745, -200.992)$ mm

10.1 A/B Pair로 각도 계산

FORMULA · CALCULATION

$dx = 150.249 - (-149.740) \approx 299.989$ mm

$dy = 200.301 - 197.683 \approx 2.618$ mm

Theta = atan2(2.618, 299.989)
 $\approx +0.500^\circ$

10.2 A/B Pair 중점

FORMULA · CALCULATION

$M_{AB} \approx ((-149.740+150.249)/2,$
 $(197.683+200.301)/2)$
 $\approx (0.255, 198.992)$ mm

설계 Pair 중점 (0, 200) 을 0.5° 회전하면 약 (-1.745, 199.992) 가 된다.

FORMULA · CALCULATION

Glass Center T
 $= M_AB - R(0.5^\circ) \times (0, 200)$
 $\approx (0.255, 198.992) - (-1.745, 199.992)$
 $\approx (+2.000, -1.000) \text{ mm}$

10.3 Mark C로 독립 검산

FORMULA · CALCULATION

$R(0.5^\circ) \times (0, -200) \approx (+1.745, -199.992)$

$T_C = C' - R(0.5^\circ) \times (0, -200)$
 $\approx (+3.745, -200.992) - (+1.745, -199.992)$
 $\approx (+2.000, -1.000) \text{ mm}$

A/B와 C가 같은 Glass 중심을 가리키므로 Geometry가 일관적이다.

11. 자동 Align 실행 로직

11.1 전체 흐름

TEXT

1. Align Recipe Load
↓
2. Mark A 예상 위치로 이동 → 촬영 → 중심 검출
↓
3. Mark B 예상 위치로 이동 → 촬영 → 중심 검출
↓
4. Mark C 예상 위치로 이동 → 촬영 → 중심 검출
↓
5. 세 Mark Pixel 좌표를 공통 Machine 좌표로 변환
↓
6. Mark ID/Score/거리/Triangle Geometry 검증
↓
7. 3점 Rigid Fit으로 Glass Center X/Y/Theta 계산
↓
8. Mark별 Residual 계산 및 허용 오차 검사
↓
- 9A. Physical: Stage X/Y/Theta 보정
또는
- 9B. Virtual: ROI/검사 좌표 Transform
↓
10. 필요 시 재촬영하여 최종 오차 확인
↓
11. Align Result와 Image 저장

11.2 Coarse-to-Fine Search

초기 Glass 위치 편차가 크다면:

1. 넓은 Search ROI와 낮은 배율/Coarse Parameter로 Mark를 찾는다.
2. 예상 위치를 갱신한다.
3. 작은 ROI와 Sub-pixel 방법으로 Mark 중심을 정밀 측정한다.
4. Score와 주변 유사 Pattern을 검증한다.

이 방식은 전체 Image를 매번 정밀 Matching하는 것보다 빠르고 오검출을 줄이기 쉽다.

11.3 Stage 보정 시 주의

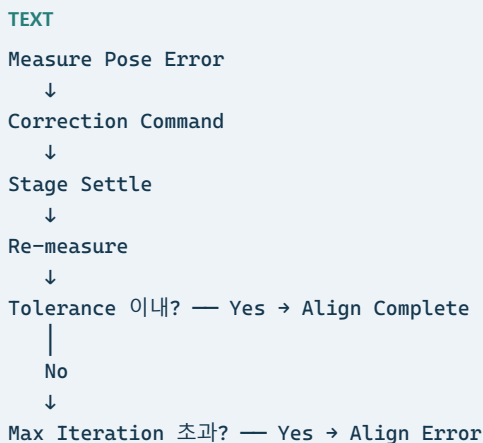
계산된 Glass Error가 $(+2 \text{ mm}, -1 \text{ mm}, +0.5^\circ)$ 라고 해서 장비에 항상 단순히 $(-2, +1, -0.5^\circ)$ 를 입력하면 되는 것은 아니다.

이유:

- Stage 회전 Pivot이 Glass 중심과 다를 수 있음
- X/Y 명령이 Stage Local인지 Machine Fixed 좌표인지 다름
- Rotation 후 Translation의 의미가 달라짐
- Camera가 Stage 위에 있는지 고정 Frame인지 다름

정확한 보정은 Stage Command의 좌표 정의와 Pivot Calibration을 사용해 현재 Pose의 역변환을 적용한다. 처음에는 작은 Correction 후 재촬영하는 Closed-loop 방식으로 검증한다.

11.4 반복 Align



무한 반복하지 않도록 최대 반복 횟수와 최소 개선량을 둔다.

12. Mark 3개를 어떻게 역할 분담할까?

권장 방식은 세 Mark 모두로 X/Y/Theta를 동시에 구하는 Rigid Fit이다. 이해를 위해 역할을 나누면 다음처럼 볼 수 있다.

- A/B의 긴 X Baseline: 회전각에 강함
- A/B 중점: 상단 기준 위치 제공
- C: Y 방향 Geometry와 Glass 중심 계산
- 세 Point 전체: Translation/Rotation Noise 평균화와 Residual 검사

“A/B로 Theta만 계산하고 C로 Y만 계산”하는 순차 Logic도 만들 수 있지만 측정 Noise가 분리되어 일관성이 떨어질 수 있다. 세 Point를 한 Transform으로 Fit하고 각 Pair 결과는 진단용으로 사용하는 편이 일반적으로 깔끔하다.

13. Mark 간 거리가 회전 정밀도에 미치는 영향

두 Mark의 중심 검출 오차 표준편차를 각각 σ 라고 하면 작은 오차에서 회전각 Noise는 대략 다음 경향을 가진다.

FORMULA · CALCULATION

$$\sigma_{\theta} \approx \sqrt{2} \times \sigma_{\text{position}} / \text{Mark Baseline}$$

Mark 중심 반복성이 $5 \mu\text{m} = 0.005 \text{ mm}$, Baseline이 300 mm라면:

FORMULA · CALCULATION

$$\begin{aligned} \sigma_{\theta} &\approx \sqrt{2} \times 0.005 / 300 \\ &\approx 0.00002357 \text{ rad} \\ &\approx 0.00135^{\circ} \end{aligned}$$

Baseline이 50 mm라면 약 0.0081° 로 악화된다. 따라서 Mark는 가능한 한 멀리 배치하는 것이 회전 정밀도에 유리하다. 다만 Lens Distortion, Stage Travel, Glass 변형과 FOV 접근성도 고려한다.

14. 3개 Mark가 정말 필요한가?

2개 Mark

Rigid Body라는 전제에서 X/Y/Theta를 계산할 수 있다.

장점:

- 촬영 수와 Cycle Time 감소
- Logic 단순

한계:

- 한 Mark 오검출 시 검증할 여분이 없음
- Scale/Geometry 이상을 확인하기 어려움

3개 Mark

권장되는 이유:

- 과잉 결정(Over-determined)으로 Noise 평균화
- Pair 거리와 Triangle Shape 검증
- 하나의 잘못된 측정을 Residual로 탐지
- Glass 비정상/Calibration 이상 진단

3개보다 많은 Mark

큰 Glass에서 지역 변형, Camera/Stage Calibration, 여러 검사 영역의 Local Align을 확인할 때 유리하다. Cycle Time과 관리 복잡도가 증가한다.

15. Rigid, Similarity, Affine 중 무엇을 사용할까?

Rigid Transform — 기본 권장

TEXT

X/Y Translation + Rotation

Glass가 강체이고 Camera/Stage Calibration이 정상이라면 Align은 Rigid가 기본이다.

Similarity Transform

TEXT

Translation + Rotation + Uniform Scale

온도/공정으로 Glass Scale 변화가 의미 있게 있고 검사에서 이를 모델링해야 할 때 고려한다. 단, Align이 Calibration 오류를 Scale로 숨기지 않도록 주의한다.

Affine Transform

TEXT

Translation + Rotation + X/Y Scale + Shear

Camera Perspective, Stage 직교도 오류 또는 실제 기판 변형을 흡수할 수 있지만 물리적 원인을 숨길 위험이 있다. 단순 Glass X/Y/Theta Align에는 과도할 수 있다.

[!IMPORTANT]

Transform 자유도를 늘리면 Residual은 작아지기 쉽지만 “장비가 정확해진 것”은 아니다. Glass는 강체라는 물리 모델에 맞는 최소 자유도를 사용한다.

16. 자동 Align Recipe에 저장할 항목

영역	저장 항목
Mark 정의	ID, Template, Polarity, 중심 정의, 예상 크기
설계 Geometry	Glass Size, 중심 기준 Mark X/Y 좌표
Search	예상 Stage Position, Search ROI, Search Range
Camera	Camera ID, Exposure, Gain, Light Recipe
Calibration	Pixel→Machine Transform ID/Version
Stage	Rotation Pivot, Axis Direction, Unit, Settle 조건
판정	Match Score, Position/Angle/Residual 허용값
반복	Max Iteration, 최소 개선량, Timeout
결과 저장	Mark Image, 좌표, Score, X/Y/Theta, Residual

17. 자동 Align 실패 상황과 대응

최초 Search에서 아무 Mark도 보이지 않음

- Nominal Loading Position과 Drawing 좌표가 Camera FOV 안으로 연결되는지 확인
- Glass Edge/Corner 기반 Coarse Align 또는 넓은 FOV Camera 사용
- 제한된 Raster/Spiral Search 범위와 Timeout 설정
- 작업자 Teaching 위치를 초기 Seed로 저장
- FOV 밖 Mark를 Image Processing Parameter 변경으로 찾으려 하지 않음

Mark를 찾지 못함

- Search ROI 확대 전 제품 범위와 충돌 여부 확인
- Exposure/조명 상태 확인
- Coarse Search 또는 Edge/Blob Backup 검토
- 찾지 못한 Mark를 임의 좌표로 대체하지 않음

잘못된 Mark를 찾음

- 예상 위치와 이동 허용 범위 확인
- Mark별 고유 ID 또는 Template 사용
- Pair 거리와 Triangle Geometry 검증
- 3점 Rigid Fit Residual 검사

두 Mark 각도와 세 번째 Mark가 맞지 않음

- Camera별/Stage Pose Calibration 오류
- 잘못된 Mark 순서
- Glass 방향 반전 또는 180° 투입
- Glass/Mark 파손
- Stage Backlash 또는 Encoder 문제

Align 후에도 재측정 오차가 큼

- Rotation Pivot 보정 오류
- Stage Local/Global 좌표 혼동
- Correction 순서 문제
- Stage Settle/진동
- Calibration Scale/Rotation 오류

18. 흔한 오해

Pair가 Glass 중심을 지나지 않으면 설계 Offset을 회전해 보정해야 한다.

1. **“A/B 중점이 Glass 중심이다.”**

Mark 중심과 Glass 중심 사이의 설계 Geometry가 필요하다.

2. **“Mark Image만 등록하면 Glass 중심을 안다.”**

여러 촬영 위치의 Image는 공통 Machine 좌표로 먼저 변환해야 한다.

3. **“Image Pixel 좌표끼리 빼면 바로 각도가 나온다.”**

각 촬영 중 Stage를 개별 이동하면 Glass Pose가 바뀔 수 있다. 측정 Pose와 보정 Pose를 분리해야 한다.

4. **“두 Mark를 각각 중앙에 맞추면 자동으로 Align된다.”**

강체 Glass에는 Rigid Fit이 물리적으로 맞고 Affine은 오류를 숨길 수 있다.

5. **“3점이면 Affine을 사용하는 것이 더 정확하다.”**

Rotation Pivot과 Stage 좌표 Command 의미를 반영한 역변환이 필요하다.

6. **“계산 Error에 음수만 붙여 Stage에 명령하면 된다.”**

초기 개발과 정밀 Align에서는 Closed-loop 재확인으로 보정 모델을 검증해야 한다.

7. **“한 번 보정했으면 재촬영할 필요가 없다.”**

원·십자·사각형처럼 Geometry로 정의할 수 있는 Mark는 Template 없이도 검출할 수 있다.

8. **“Template Image가 없으면 Align 자체가 불가능하다.”**

Mark 정의, 예상 위치, Geometry 또는 학습/Template 기준 없이 특정 Pattern을 Align Mark로 식별할 수는 없다.

9. “Template가 없더라도 AI가 Image에서 알아서 Align Mark를 고른다.”

원 중심, 선 교점, Corner 중심 또는 회전 Pattern Matching으로 현재 중심을 측정할 수 있다.

10. “Glass가 회전하면 Mark 중심도 정의할 수 없다.”

19. 면접 및 실무 확인 질문

Q1. Align과 Calibration의 차이는?

핵심 답변 Calibration은 Image Pixel과 Machine 좌표계 사이의 지속적인 관계를 정하고, Align은 그 Calibration을 사용해 매 Glass의 현재 X/Y/Theta 편차를 계산한다.

Q2. 2개 Mark로 X/Y/Theta를 계산할 수 있는가?

핵심 답변 Rigid Body 전제라면 가능하다. Pair Vector로 Theta를 구하고 한 Point 또는 Pair 중점과 설계 Offset으로 Translation을 구한다. 세 번째 Mark는 Noise 평균화와 오류 검증에 사용한다.

Q3. 왜 Mark를 멀리 배치하는 것이 좋은가?

핵심 답변 같은 위치 측정 오차에서 Baseline이 길수록 각도 오차가 작아지기 때문이다. 다만 Stage 범위, 왜곡과 Glass 변형을 함께 고려한다.

Q4. 왜 3개 Mark를 한 번에 Rigid Fit하는가?

핵심 답변 특정 Pair 하나에 의존하지 않고 세 측정의 Noise를 평균화하며, Mark별 Residual로 오검출과 Geometry 이상을 확인할 수 있기 때문이다.

Q5. Camera 한 대로 Mark를 순차 촬영할 때 가장 중요한 것은?

핵심 답변 각 촬영의 Stage/Camera Pose와 Image Pixel Offset을 결합하여 모든 Mark 중심을 동일한 Machine 좌표계로 변환하는 것이다.

20. 8일차 실습 문제

실습 1. 본인 장비 좌표계 그리기

다음은 한 그림에 표시한다.

- Glass 중심과 370×470 외곽
- Mark A/B/C의 실제 Drawing 좌표
- Image +x/+y
- Machine +X/+Y/+Theta
- Stage 회전 Pivot
- Camera 이동 방향 또는 Glass 이동 방향

실습 2. Pair 공식 손 계산

Mark Pair 기준 좌표와 현재 측정 좌표를 정해 다음을 계산한다.

- Reference Pair Angle
- Current Pair Angle
- DeltaTheta
- Pair Midpoint
- Pair Midpoint에서 Glass Center까지의 회전된 Offset

실습 3. Align Recipe 설계

Recipe 표를 기준으로 현재 장비에서 이미 알고 있는 값과 아직 모르는 값을 나눈다.

실습 4. 실패 판정표 작성

다음 상황의 처리 정책을 정한다.

- Mark 1개 Match 실패
- 3개 Match 성공이지만 한 Mark Residual 초과
- Mark 간 거리 0.2 mm 변화
- Align 보정 후 오차가 오히려 증가
- Max Iteration 초과

21. 권장 구현 순서

코드를 작성하기 전에 다음 순서로 작은 검증을 한다.

1. 고정된 기준 Glass에서 Mark 중심 반복 측정 정밀도를 확인한다.
2. Mark마다 Image Pixel을 Machine 좌표로 변환하는 결과를 확인한다.
3. 사람이 Glass를 X/Y로 이동했을 때 계산 Center가 같은 양만큼 움직이는지 확인한다.
4. Glass를 알려진 각도로 회전시켜 계산 Theta를 비교한다.
5. 3점 Rigid Fit Residual과 Pair별 Theta를 비교한다.
6. Stage 보정 없이 Virtual ROI Transform부터 검증한다.
7. 작은 Physical Correction을 적용하고 재촬영한다.
8. Rotation Pivot Compensation을 검증한다.
9. 반복 Align과 실패 정책을 추가한다.
10. Recipe/Result/Review 저장을 연결한다.

22. 핵심 결론

- ● Mark 3개의 Image를 등록하는 것보다 Mark와 Glass 중심 사이 Geometry를 등록하는 것이 핵심이다.
- ● 모든 Mark 중심을 먼저 하나의 Machine 좌표계로 변환해야 한다.
- ● 수평 Pair의 각도는 $\text{atan2}(dy, dx)$, 수직 Pair는 그 값에서 기준 90°를 뺀다.
- ● Pair 중점이 Glass 중심이 아니라면 회전된 설계 Offset을 보정해야 한다.
- ● 권장 방식은 3개 Mark 전체의 2D Rigid Fit으로 Center X/Y/Theta를 동시에 계산하는 것이다.

- 세 번째 Mark는 Residual, 거리, Triangle Geometry를 통해 오검출과 Calibration 이상을 찾는다.
- Stage 보정에는 Rotation Pivot과 좌표 Command 정의를 반영한 역변환이 필요하다.
- 최초 개발에서는 보정 후 재촬영하는 Closed-loop Align으로 정확성을 검증한다.
- Align Result에는 Mark별 좌표/Score/Image, Center X/Y/Theta와 Residual을 저장한다.
- 큰 Glass에서는 Global Align 후 검사 영역별 Local Align이 추가로 필요할 수 있다.
- 실제 Template Image는 검출 방식에 따라 선택 사항이지만 Mark의 사전 정의는 필수다.
- 최초 Image가 없으면 CAD/Nominal 위치와 Geometry Finder 또는 작업자 Teaching으로 첫 Template를 만든다.
- Glass가 회전되어도 회전 허용 검색 또는 Geometry Fit으로 각 Mark 중심을 먼저 측정할 수 있다.

8일차 학습 완료 체크

- ☐ Glass 중심 기준으로 Mark A/B/C의 실제 좌표를 적었다.
- ☐ Image 좌표와 Machine 좌표의 축 방향을 그렸다.
- ☐ 수평/수직 Mark Pair로 Theta를 계산할 수 있다.
- ☐ Pair 중점에서 Glass 중심을 구할 때 Offset 회전이 필요한 이유를 안다.
- ☐ 3점 Rigid Fit의 입력과 출력, Residual 의미를 설명한다.
- ☐ Camera 한 대의 순차 촬영 좌표를 Machine 좌표로 합치는 방법을 이해한다.
- ☐ Align Recipe와 자동 실행 상태 흐름을 설계했다.
- ☐ Rotation Pivot을 포함한 Stage 보정 검증 계획을 세웠다.
- ☐ 내 Mark가 Template 방식인지 Geometry 방식인지 선택하고 근거를 적었다.
- ☐ 최초 Template가 없을 때 Mark를 FOV 안으로 가져오는 Coarse Search 방법을 정했다.

12일차. ROI와 Align 기반 Coordinate Transform

Phase 8 · 예상 학습 시간: 10~14시간 · 난이도: 중급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

핵심 개념

- ROI의 목적과 Static/Dynamic ROI
- Align 결과 X/Y/Theta를 ROI에 적용하는 방법
- 회전 중심(Pivot)과 Transform 순서
- Rect, RotatedRect, Polygon, Mask의 차이
- Reference Coordinate와 Current Image Coordinate
- ROI Clip/Out-of-image 처리
- Search ROI와 Inspection ROI 분리
- Image Warp와 ROI Point Transform의 선택

실무 목표

Recipe에 기준 좌표로 저장된 ROI를 제품의 현재 Align Pose에 맞춰 이동·회전한다. Axis-aligned Rect를 억지로 회전시키지 않고 Polygon/RotatedRect/Mask를 사용하며, Transform Pivot·좌표 방향·경계 오류를 명확히 처리한다.

2. 선수 지식

- 8일차의 Rigid Transform과 Glass 중심
- Image Coordinate (x, y) 와 Machine Coordinate
- 삼각함수와 2×2 Rotation Matrix

FORMULA · CALCULATION

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

OpenCV Image는 보통 아래쪽이 +y이므로 수학 좌표의 CCW 부호와 화면상 회전 방향이 다르게 보일 수 있다. Align Module과 ROI Module의 부호 계약을 Test로 고정한다.

3. 핵심 개념

3.1 ROI를 왜 사용하는가?

- 검사 위치를 명시
- 계산량과 False Positive 감소
- 검사 항목별 Parameter 분리
- Review Overlay와 Recipe Editing 제공

ROI는 단순 Crop이 아니라 **제품 좌표계에 정의된 검사 Geometry**다.

3.2 Static ROI와 Dynamic ROI

- Static ROI: Image의 고정 Pixel 위치. 제품 위치가 반복 정밀도 안에서 고정될 때 사용.
- Dynamic ROI: Reference ROI에 Align Transform을 적용. 제품 X/Y/Theta 변화에 대응.

TEXT

Reference Product ROI --Align Pose--> Current Image ROI

3.3 Search ROI와 Inspection ROI

- Align Search ROI: Mark가 이동할 가능성을 포함하는 넓은 영역
- Inspection ROI: Align 후 실제 검사할 좁은 영역

Inspection ROI로 Mark를 먼저 찾으려 하면 제품 편차가 ROI보다 큰 순간 Align이 실패한다.

3.4 기본 Rigid Transform

Reference Point $p=(x,y)$ 를 Current Point로 변환:

FORMULA · CALCULATION

$$p_{\text{current}} = R(\theta) \times p_{\text{reference}} + t$$

$$t = (tx, ty)$$

회전 중심 c 가 별도로 주어지면:

FORMULA · CALCULATION

$$p_{\text{current}} = R(\theta) \times (p_{\text{reference}} - c) + c + t$$

Glass 중심 또는 Align 기준점을 Pivot으로 사용하는 경우가 일반적이다.

3.5 Transform 순서가 중요한 이유

TEXT

Rotate about origin → Translate

와

TEXT

Translate → Rotate about origin

은 결과가 다르다. Recipe에서 Align X/Y가 “회전 전 Reference Frame의 Translation”인지 “회전 후 Current Frame의 Translation”인지 인터페이스로 고정한다.

3.6 ROI 표현

표현	회전	Mask	장점	한계
<code>cv::Rect</code>	불가	사각	빠른 Crop	회전 ROI 표현 불가
<code>cv::RotatedRect</code>	가능	사각	Center/Size/Angle 직관적	복잡 형상 불가
Polygon	가능	가능	임의 형상, Point Transform	Raster Mask 생성 필요
Binary Mask	가능	직접	Pixel 선택에 편리	Transform 보간/관리 필요

회전된 ROI의 Axis-aligned Bounding Rect만 검사하면 원하지 않는 Background까지 포함된다. Bounding Rect로 Crop한 뒤 Polygon Mask를 적용한다.

3.7 ROI를 움직일까, Image를 움직일까?

ROI Transform

원본 Image는 유지하고 ROI Point만 Current Pose로 변환한다.

- 보간에 의한 Image 변화 없음
- 여러 ROI에 효율적
- 각 알고리즘이 회전 ROI를 처리해야 함

Image Warp/Rectification

Current Image를 Reference 좌표로 역변환한다.

- 모든 ROI를 Reference 위치에서 처리 가능
- 구현이 단순해질 수 있음
- 보간, Border, 처리 시간과 측정 Bias 발생

정밀 Edge 측정은 가능한 한 원본 Image 좌표에서 수행하고 좌표만 Transform하는 방식을 우선 검토한다.

3.8 Dynamic RotatedRect

Reference RotatedRect가 Center $\mathbf{r}$, Size $(\mathbf{w}, \mathbf{h})$, Angle α 라면 Rigid Align 후:

FORMULA · CALCULATION

$$\text{center}' = R(\theta)(\mathbf{r}-\mathbf{c})+\mathbf{c}+\mathbf{t}$$

$$\text{size}' = (\mathbf{w}, \mathbf{h})$$

$$\text{angle}' = \alpha + \theta$$

Rigid Transform에서는 Size가 변하지 않는다. Scale까지 바뀐다면 Similarity/Affine 모델의 사용 근거를 별도로 확인한다.

3.9 ROI 경계 처리

Transform 후 ROI 일부가 Image 밖이면 선택지는 다음과 같다.

- Align/Inspection Error로 처리
- 허용 Margin 이내만 Clip하고 `partial=true` 기록
- Border Padding 후 처리
- Search 범위/Camera FOV 재설계

치수/Area 검사에서 잘린 ROI를 정상 처리하면 안 된다. Clip을 숨기지 말고 Result 상태로 남긴다.

3.10 좌표 반올림

Transform Point는 `double/float` 로 유지한다. Crop용 Bounding Rect를 만들 때:

- Left/Top: `floor`
- Right/Bottom: `ceil`

을 사용해 Polygon 전체가 포함되도록 한다. 초기 정수 반올림은 반복 Transform과 측정에 Bias를 만든다.

4. 그림으로 이해하기



TEXT

ROI Polygon Points → Rigid Transform → Bounding Rect Crop
+ Polygon Mask

5. 실제 검사 장비에서 어디에 사용하는가?

- Align 후 Mark/Pad/Particle 검사 ROI 이동
- Glass의 여러 검사 영역을 Global Align Pose로 배치
- Local Mark로 특정 영역만 추가 보정
- Review Overlay를 현재 Image에 표시
- 검사 결과 좌표를 Reference 제품 좌표로 역변환

큰 Glass는 Global Rigid Align만으로 지역 변형을 모두 설명하지 못할 수 있다. Global ROI 배치 후 Local Align/Calibration Residual을 확인한다.

6. 숫자로 이해하기

예제 1: ROI 중심을 Pivot 자체로 회전

Reference ROI Center (1000,800) , Align tx=+12.5 , ty=-3.2 , $\theta=+0.15^\circ$ 이고 Pivot이 ROI Center라면:

FORMULA · CALCULATION

Current Center = (1012.5,796.8)

Current Angle = Reference Angle + 0.15°

Pivot 위 Point는 Rotation으로 이동하지 않는다.

예제 2: Glass 중심 Pivot

Pivot c=(1024,1024) , ROI Center r=(1000,800) :

FORMULA · CALCULATION

$$r-c = (-24, -224)$$

$$\theta = 0.15^\circ \approx 0.00261799 \text{ rad}$$

$$R(\theta)(r-c) \approx (-23.4135, -224.0621)$$

$$\begin{aligned} \text{center}' &\approx (1024, 1024) + (-23.4135, -224.0621) + (12.5, -3.2) \\ &\approx (1013.0865, 796.7379) \end{aligned}$$

단순히 **(1012.5, 796.8)** 로 이동한 값과 다르다. Pivot에서 멀수록 작은 각도도 위치 차이를 만든다.

예제 3: 회전으로 인한 이동 크기

Pivot에서 250 mm 떨어진 ROI가 **0.1°** 회전하면 Arc 이동 근사:

FORMULA · CALCULATION

$$s = r\theta = 250 \times (0.1\pi/180) \approx 0.4363 \text{ mm}$$

큰 Glass에서는 매우 작은 Theta도 ROI 위치에 큰 영향을 준다.

예제 4: Bounding Rect 반올림

변환 Polygon x 범위가 **100.2~300.7** , y 범위가 **50.8~180.1** 이면:

FORMULA · CALCULATION

$$\text{left} = \text{floor}(100.2) = 100$$

$$\text{top} = \text{floor}(50.8) = 50$$

$$\text{right} = \text{ceil}(300.7) = 301$$

$$\text{bottom} = \text{ceil}(180.1) = 181$$

$$\text{Rect size} = 201 \times 131$$

7. C++ 구현

CPP · 1/2

```
#include <opencv2/opencv.hpp>

#include <cmath>
#include <stdexcept>
#include <vector>

struct Pose2d final {
    double txPixels{};
    double tyPixels{};
    double thetaDegrees{};
};

[[nodiscard]] cv::Point2d TransformPoint(
    const cv::Point2d& point,
    const cv::Point2d& pivot,
    const Pose2d& pose)
{
    if (!std::isfinite(pose.txPixels) || !std::isfinite(pose.tyPixels) ||
        !std::isfinite(pose.thetaDegrees)) {
        throw std::invalid_argument{"Pose must be finite"};
    }
    constexpr double pi=3.14159265358979323846;
    const double radians=pose.thetaDegrees*pi/180.0;
    const double c=std::cos(radians), s=std::sin(radians);
    const cv::Point2d q=point-pivot;
    return {c*q.x-s*q.y+pivot.x+pose.txPixels,
            s*q.x+c*q.y+pivot.y+pose.tyPixels};
}

[[nodiscard]] std::vector<cv::Point2d> TransformPolygon(
    const std::vector<cv::Point2d>& polygon,
    const cv::Point2d& pivot,
    const Pose2d& pose)
{
    if (polygon.size()<3) {
        throw std::invalid_argument{"ROI polygon needs at least 3 points"};
    }
    std::vector<cv::Point2d> result;
    result.reserve(polygon.size());
    for (const auto& p:polygon) result.push_back(TransformPoint(p,pivot,pose));
    return result;
}
```


CPP · 2/2

```
[[nodiscard]] cv::Rect SafeBoundingRect(
    const std::vector<cv::Point2d>& polygon,
    const cv::Size& imageSize)
{
    if (polygon.empty() || imageSize.width<=0 || imageSize.height<=0) {
        throw std::invalid_argument{"Invalid polygon or image size"};
    }
    double minX=polygon[0].x,maxX=polygon[0].x;
    double minY=polygon[0].y,maxY=polygon[0].y;
    for (const auto& p:polygon) {
        minX=std::min(minX,p.x); maxX=std::max(maxX,p.x);
        minY=std::min(minY,p.y); maxY=std::max(maxY,p.y);
    }
    const cv::Rect requested{
        static_cast<int>(std::floor(minX)),
        static_cast<int>(std::floor(minY)),
        static_cast<int>(std::ceil(maxX)-std::floor(minX)),
        static_cast<int>(std::ceil(maxY)-std::floor(minY))};
    const cv::Rect imageBounds{0,0,imageSize.width,imageSize.height};
    const cv::Rect clipped=requested & imageBounds;
    if (clipped != requested) {
        throw std::out_of_range{"Transformed ROI leaves the image"};
    }
    return requested;
}
```

Unit Test

CPP

```
#include <cassert>

void TestGivenAlignExample()
{
    const auto p=TransformPoint({1000,800},{1024,1024},{12.5,-3.2,0.15});
    assert(std::abs(p.x-1013.0865)<0.001);
    assert(std::abs(p.y-796.7379)<0.001);
}

void TestNinetyDegreeRotation()
{
    const auto p=TransformPoint({11,10},{10,10},{0,0,90});
    assert(std::abs(p.x-10)<1e-9);
    assert(std::abs(p.y-11)<1e-9);
}
```

코드에서 봐야 할 점

1. Point와 Pose는 정수로 조기 반올림하지 않는다.
2. Pivot을 명시적 인자로 전달한다.
3. Polygon 전체를 변환한 뒤 Bounding Rect를 만든다.
4. ROI가 Image 밖이면 조용히 Clip하지 않고 Error로 드러낸다.
5. OpenCV 화면 좌표의 Angle Convention은 실제 Align 출력과 Golden Test로 확인한다.

8. 실무에서 발생하는 문제

1. **Pivot을 ROI 중심으로 잘못 사용**: Glass 중심/Align 기준점 계약 확인.
2. **Image y축과 Machine Y축 부호 혼동**: Calibration 후 좌표에서 Transform.
3. **회전 ROI를 Axis-aligned Rect로만 검사**: Polygon Mask 적용.
4. **ROI 일부가 FOV 밖인데 계속 검사**: Partial/Error 상태와 Margin 설계.
5. **Point마다 정수 반올림**: double 유지, Raster 단계에서만 floor/ceil.
6. **Global Align로 큰 Glass 전체를 가정**: 지역 Residual과 Local Align 검토.
7. **Image Warp 후 정밀 Edge 측정**: 보간 Bias를 원본 좌표 측정과 비교.

9. 흔한 오해

1. **"ROI 이동은 X/Y를 더하면 끝이다."** Theta와 Pivot이 필요하다.
2. **"Rect를 회전하면 또 Rect다."** 축 정렬 Rect가 아니라 RotatedRect/Polygon이다.
3. **"Bounding Rect가 곧 회전 ROI다."** 불필요한 Background를 포함한다.
4. **"Clip된 ROI도 같은 검사다."** 정보가 잘려 결과 의미가 달라진다.
5. **"Image를 회전하면 좌표 문제가 사라진다."** 보간과 역변환 관리가 추가된다.
6. **"0.1°는 무시할 만큼 작다."** 큰 Glass 가장자리에서는 수백 μm 이동이다.

10. 면접에서 나올 수 있는 질문

Q1. Static ROI와 Dynamic ROI의 차이는?

답변: Static은 Image 고정 좌표, Dynamic은 Reference ROI에 Align X/Y/Theta를 적용한 현재 제품 좌표다.

Q2. ROI와 Align은 어떤 관계인가?

답변: Align이 제품 Pose를 구하고 ROI Transform이 그 Pose로 검사 Geometry를 현재 Image에 배치한다.

Q3. 왜 Pivot이 중요한가?

답변: 같은 각도라도 회전 중심에서 떨어진 Point의 이동량이 달라지기 때문이다.

Q4. 회전 ROI를 어떻게 처리하는가?

답변: Polygon/RotatedRect Point를 변환하고 Bounding Crop 안에 Mask를 적용한다.

Q5. ROI가 Image 밖으로 나가면?

답변: 검사 의미에 따라 Error/Partial 정책을 명시하며 치수 검사는 일반적으로 Error 처리한다.

Q6. ROI Transform과 Image Warp 중 무엇이 좋은가?

답변: 원본 보존과 정밀 측정에는 Point/ROI Transform을 우선하고, 다수 Reference ROI 처리 편의가 중요하면 Warp를 검증 후 사용한다.

11. 실습 문제

1. Pivot $(0, 0)$, Point $(100, 50)$, $tx=10, ty=-5, \theta=1^\circ$ 를 계산한다.
2. 370×470 Glass의 네 Corner를 $\pm 1^\circ$ 회전에 필요한 FOV Margin을 계산한다.
3. Rect를 Polygon으로 만들고 X/Y/Theta 후 Mask를 생성한다.
4. ROI가 Image를 1/10/50 Pixel 벗어나는 경우의 정책을 설계한다.

Phase 8 미니 프로젝트: Dynamic ROI Viewer

Reference Image/ROI와 Align Pose를 입력해 변환 Polygon, Bounding Rect, Mask와 경계 상태를 Overlay하고 JSON/CSV로 좌표를 저장한다.

12. Chapter 핵심 요약

- ROI는 제품 기준 검사 Geometry다.
- Dynamic ROI는 $R(\theta)(p-c)+c+t$ 로 계산한다.
- Pivot과 좌표 축/부호 계약이 필수다.
- 회전 ROI는 Polygon/RotatedRect와 Mask로 처리한다.
- Point는 실수로 유지하고 Raster 단계에서만 반올림한다.
- Image 밖 ROI를 숨겨서 Clip하지 않는다.
- 정밀 측정은 원본 Image 좌표를 우선한다.
- 큰 Glass는 Global 후 Local Residual을 확인한다.

12일차 학습 완료 체크

- ☐ Static/Dynamic ROI를 구분한다.
- ☐ Pivot 포함 Point Transform을 계산한다.
- ☐ Polygon Mask와 Bounding Rect를 구분한다.
- ☐ ROI 경계 및 반올림 정책을 정했다.
- ☐ Dynamic ROI Viewer를 설계하거나 구현했다.

다음 학습 예고

13일차에는 Pattern/Template/Feature Matching, Global/Local Alignment와 3점 Rigid Fit을 심화한다.

13일차. Alignment 심화: Matching에서 X/Y/Theta까지

Phase 9 · 예상 학습 시간: 16~22시간 · 난이도: 중급→고급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

핵심 개념

- Alignment의 목적과 Global/Local Align
- Translation, Rotation, Scale과 2D Rigid Transform
- Fiducial/Align Mark의 설계 원칙
- Template, Pattern, Feature Matching의 차이
- Match Score와 Position Accuracy의 차이
- 2점/3점 이상으로 X/Y/Theta 계산
- Residual, Mark 거리와 Geometry 검증
- Pyramid/Angle Search/Sub-pixel Refinement
- RANSAC과 Outlier 제거
- Align 실패 정책과 Review Data

실무 목표

검출된 Mark 좌표를 제품 기준좌표와 대응시켜 최적 X/Y/Theta를 계산한다. Match Score만으로 성공을 판단하지 않고 Mark ID, 위치 범위, Pair 거리, Rigid Fit Residual과 재촬영 오차를 검증한다.

2. 선수 지식

- 3개 Align Mark Glass 사례
- ROI Rigid Transform
- Rotation Matrix와 `atan2`
- Image→Machine Calibration의 필요성

FORMULA · CALCULATION

$$p_{\text{current}} = R(\theta)q_{\text{reference}} + t$$

`q` 와 `p` 는 반드시 같은 단위/축 방향의 좌표여야 한다.

3. 핵심 개념

3.1 Alignment란?

Reference 제품과 현재 제품의 Pose 차이를 추정하는 과정이다.

TEXT

Reference Geometry + Current Image Features

↓

X / Y / Theta

↓

Stage Correction 또는 ROI Transform

Alignment는 Calibration과 다르다. Calibration은 좌표계 사이 관계를 구하고 Align은 그 관계를 이용해 제품마다 달라지는 Pose를 구한다.

3.2 Global과 Local Alignment

- Global Align: 제품 전체의 대표 X/Y/Theta. Glass 외곽 또는 2~3개 Global Mark 사용.
- Local Align: 특정 검사 영역 주변의 추가 위치/각도 보정. Pattern 변형, 공정 Scale, 대형 기판 지역 오차 대응.

TEXT

Global Pose → 모든 ROI 1차 배치 → Local Mark → 해당 ROI 미세 보정

Local Align으로 Global Calibration 불량을 숨기지 않는다. Local Correction 분포를 장비 상태 지표로 활용한다.

3.3 Transform 자유도

Model	자유도	최소 대응점	적용
Translation	2	1	회전 없는 위치 편차
Rigid	3	2	강체 X/Y/Theta
Similarity	4	2	Rigid+균일 Scale
Affine	6	3 non-collinear	X/Y Scale+Shear 포함
Homography	8	4 non-collinear	Perspective Plane

강체 Glass Align은 Rigid가 기본이다. 자유도를 늘리면 Residual은 줄지만 Calibration/기구 오류를 Scale-Shear로 흡수할 수 있다.

3.4 Fiducial 설계

좋은 Mark의 조건:

- Background와 높은 Contrast
- 회전/반전 ID 구분
- 주변 Pattern과 고유성
- 오염/파손에도 중심 정의가 안정적
- 가능한 긴 Baseline과 비공선 배치
- Camera FOV와 Stage 이동 범위 안
- Etching/Printing 공정 Variation이 작음

원은 중심은 안정적이지만 자체 각도를 제공하지 않는다. 여러 원의 배치 Vector로 제품 각도를 구한다.

3.5 Template Matching

Reference Patch를 Current Image의 각 위치와 비교한다.

대표 Score:

- Normalized Cross Correlation
- Squared Difference

장점: 구현과 해석이 쉬우며 고정 Pattern에 강함. 약점: 회전, Scale, 조명, 부분 가림에 민감. Angle/Scale Search가 필요하면 계산량이 증가한다.

3.6 Pattern Matching

상용 Vision Library에서 Pattern Matching은 Edge/Geometry 기반, Pyramid와 Angle Search, Sub-pixel Pose를 포함하는 더 넓은 기능을 뜻하는 경우가 많다. Library마다 의미와 Score 정의가 다르므로 API 문서를 기준으로 계약한다.

3.7 Feature Matching

ORB/SIFT 계열 Keypoint와 Descriptor를 비교한다.

장점:

- 회전/Scale/부분 가림에 상대적으로 강함
- 넓은 Scene의 대응점 제공

약점:

- 단순/반복 Pattern에서 불안정
- Outlier와 잘못된 대응
- 정밀 반복성은 전용 Mark/Pattern Matcher보다 낮을 수 있음

검사 장비의 정밀 Align에는 명확한 Fiducial/Geometry가 우선이며 Feature Matching은 복잡한 Texture나 Coarse Align에 검토한다.

3.8 Coarse-to-Fine

TEXT

넓은 ROI + Image Pyramid + 넓은 Angle

↓

Coarse Pose

↓

작은 ROI + 원본 해상도 + 좁은 Angle

↓

Sub-pixel Pose

처리 시간과 오검출을 함께 줄인다. Coarse 단계 실패 시 Fine Search를 억지로 실행하지 않는다.

3.9 Match Score는 정확도가 아니다

높은 Score는 Pattern 유사도가 높다는 의미이지 위치가 정확하다는 보장이 아니다.

- 반복 Pattern의 잘못된 Peak
- Blur 상태의 넓은 Peak
- 대칭 Mark의 180° 모호성
- 부분 가림에도 높은 배경 상관

Score와 함께 Peak Separation, Expected Position, Angle, Mark Geometry, Residual을 확인한다.

3.10 2점 Align

Reference q_1, q_2 , Current p_1, p_2 :

FORMULA · CALCULATION

$$\theta = \text{angle}(p_2 - p_1) - \text{angle}(q_2 - q_1)$$

$$t = p_1 - R(\theta)q_1$$

두 점 거리도 비교한다.

FORMULA · CALCULATION

$$\text{scale check} = |p_2 - p_1| / |q_2 - q_1|$$

Rigid 모델이면 1에 가까워야 한다.

3.11 N점 Rigid Least Squares

Centroid를 뺀 좌표 q' , p' 에서:

FORMULA · CALCULATION

$$A = \sum (q'_x p'_x + q'_y p'_y)$$

$$B = \sum (q'_x p'_y - q'_y p'_x)$$

$$\theta = \text{atan2}(B, A)$$

$$t = \bar{p} - R(\theta)\bar{q}$$

각 Point Residual:

FORMULA · CALCULATION

$$e_i = |p_i - (R q_i + t)|$$

$$RMS = \sqrt{\sum e_i^2 / N}$$

3.12 RANSAC

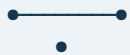
대응점이 많고 Outlier가 있을 때 일부 Point로 Model을 반복 생성하고 Inlier가 가장 많은 Model을 선택한다. 세 개뿐인 중요한 Align Mark에서는 Mark별 Geometry/ID 검증과 명시적 실패가 더 적합할 수 있다. 잘못된 Mark를 자동 제외한 사실을 숨기지 않는다.

3.13 Align Result에 포함할 데이터

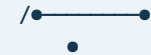
- X/Y/Theta와 좌표계/단위
- Mark별 Image/Machine 좌표와 Score
- Reference/Current Pair 거리
- Mark별 Residual, RMS/Max Residual
- Search ROI와 사용 Algorithm Version
- Calibration/Recipe Version
- 처리 시간, 반복 횟수, 실패 사유

4. 그림으로 이해하기

TEXT

Reference marks q_i 

→

Current detections p_i 

Correspondence → Rigid Fit → X/Y/Theta → Residual Check

TEXT

Match success

- └ Score pass?
- └ Expected ROI/angle pass?
- └ Mark ID/geometry pass?
- └ Pair distance pass?
- └ Fit residual pass? → Align Success

5. 실제 검사 장비에서 어디에 사용하는가?

- Glass/Panel Global Pose 보정
- Pad/Mark 주변 Local ROI 배치
- Stage X/Y/Theta Physical Correction
- Review Overlay와 NG 좌표의 Reference 변환
- Multi-camera 측정 결과를 제품 좌표로 통합

제품이 $X \pm 20$ Pixel, $Y \pm 15$ Pixel, $\text{Rotation} \pm 1^\circ$ 로 움직이면 고정 ROI는 Edge/Defect 위치를 잘못 측정한다. Align Pose로 ROI를 이동·회전해야 한다.

6. 숫자로 이해하기

예제 1: 2점 각도

FORMULA · CALCULATION

Reference: $q1=(0,0)$, $q2=(300,0)$

Current: $p1=(10,-5)$, $p2=(309.954,0.236)$

Current vector $\approx (299.954, 5.236)$

$\theta = \text{atan2}(5.236, 299.954) \approx 1.000^\circ$

$t=(10,-5)$

예제 2: Baseline과 각도 Noise

Point 반복성 $\sigma=0.005$ mm 일 때:

FORMULA · CALCULATION

$\sigma\theta \approx \sqrt{2}\sigma/L$

$L=300$ mm $\rightarrow 0.00135^\circ$

$L=50$ mm $\rightarrow 0.00810^\circ$

예제 3: Residual

세 Mark Residual이 $0.012, 0.018, 0.030$ mm 이면:

FORMULA · CALCULATION

$$\text{RMS} = \sqrt{((0.012^2 + 0.018^2 + 0.030^2)/3)}$$

$$\approx 0.02136 \text{ mm}$$

$$\text{Max} = 0.030 \text{ mm}$$

RMS와 Max를 모두 사용하면 한 Mark만 크게 벗어난 상황을 찾기 쉽다.

예제 4: 큰 Glass 가장자리 오차

중심에서 250 mm 떨어진 ROI에 Theta Error 0.02° 가 남으면:

FORMULA · CALCULATION

$$\text{arc} \approx 250 \times 0.02\pi / 180 \approx 0.0873 \text{ mm} = 87.3 \text{ } \mu\text{m}$$

각도 허용값은 Glass 크기와 ROI 위치 오차 요구로 역산한다.

7. C++ 구현

CPP · 1/2

```
#include <opencv2/opencv.hpp>

#include <algorithm>
#include <cmath>
#include <stdexcept>
#include <vector>

struct RigidPose final { double tx{},ty{},thetaDegrees{},rmsResidual{},maxResidual{}; };

[[nodiscard]] RigidPose EstimateRigidPose(
    const std::vector<cv::Point2d>& reference,
    const std::vector<cv::Point2d>& measured)
{
    if(reference.size()!=measured.size()||reference.size()<2)
        throw std::invalid_argument{"At least two corresponding points required"};
    cv::Point2d qbar{},pbar{};
    for(size_t i=0;i<reference.size();++i){qbar+=reference[i];pbar+=measured[i];}
    qbar*=1.0/reference.size(); pbar*=1.0/measured.size();

    double a=0,b=0;
    for(size_t i=0;i<reference.size();++i){
        const auto q=reference[i]-qbar, p=measured[i]-pbar;
        a+=q.x*p.x+q.y*p.y;
        b+=q.x*p.y-q.y*p.x;
    }
    if(std::hypot(a,b)<1e-12) throw std::runtime_error{"Degenerate mark geometry"};
    const double theta=std::atan2(b,a), c=std::cos(theta),s=std::sin(theta);
    const cv::Point2d rqbar{c*qbar.x-s*qbar.y,s*qbar.x+c*qbar.y};
    const cv::Point2d translation=pbar-rqbar;

    double sumSquared=0,maxResidual=0;
    for(size_t i=0;i<reference.size();++i){
        const auto& q=reference[i];
        const cv::Point2d predicted{c*q.x-s*q.y+translation.x,
                                     s*q.x+c*q.y+translation.y};
        const double residual=cv::norm(measured[i]-predicted);
        sumSquared+=residual*residual;
        maxResidual=std::max(maxResidual,residual);
    }
}
```

CPP · 2/2

```
constexpr double pi=3.14159265358979323846;
return {translation.x,translation.y,theta*180.0/pi,
        std::sqrt(sumSquared/reference.size()),maxResidual};
}

struct TemplateHit final { cv::Point2d center; double score{}; };

[[nodiscard]] TemplateHit MatchTranslationOnly(
    const cv::Mat& searchImage,const cv::Mat& templ)
{
    if(searchImage.empty()||templ.empty()||searchImage.type()!=templ.type()||
        templ.cols>searchImage.cols||templ.rows>searchImage.rows)
        throw std::invalid_argument{"Invalid template/search image"};
    cv::Mat response;
    cv::matchTemplate(searchImage,templ,response,cv::TM_CCORR_NORMED);
    double minValue=0,maxValue=0; cv::Point minLocation,maxLocation;
    cv::minMaxLoc(response,&minValue,&maxValue,&minLocation,&maxLocation);
    return {{maxLocation.x+(templ.cols-1)/2.0,
            maxLocation.y+(templ.rows-1)/2.0},maxValue};
}
```

Unit Test

CPP

```
#include <cassert>

void TestRigidPose()
{
    const std::vector<cv::Point2d> q{{0,0},{300,0},{0,200}};
    constexpr double pi=3.14159265358979323846;
    const double th=1.0*pi/180.0,c=std::cos(th),s=std::sin(th);
    std::vector<cv::Point2d> p;
    for(const auto& v:q)p.push_back({c*v.x-s*v.y+10,s*v.x+c*v.y-5});
    const auto pose=EstimateRigidPose(q,p);
    assert(std::abs(pose.tx-10)<1e-9&&std::abs(pose.ty+5)<1e-9);
    assert(std::abs(pose.thetaDegrees-1)<1e-9&&pose.rmsResidual<1e-9);
}
```

`MatchTranslationOnly` 는 회전/Scale을 처리하지 않는다. 실제 Angle Search는 Pyramid, 회전 Template/Edge Model 또는 검증된 Library를 사용하고 반환 Pose Convention을 Test한다.

8. 실무에서 발생하는 문제

1. 반복 Pattern의 잘못된 Peak → 고유 Mark, ROI, Peak Separation, Geometry 검증.
2. 대칭 Mark 각도 모호성 → Mark 배치 Vector 또는 비대칭 ID.
3. Score는 높지만 Blur로 위치 반복성 저하 → Peak Width/반복 측정.
4. Mark 한 개 오염 → Residual로 실패 처리, 임의 보간 금지.
5. Camera 순차 촬영 좌표 혼용 → 모든 Point를 Machine 좌표로 변환.
6. 자유도 과다 → Rigid Residual 원인부터 분석.
7. Stage Pivot 오류 → 보정 후 Closed-loop 재촬영.

9. 흔한 오해

1. Match Score가 높으면 위치도 정확하다.
2. Mark 한 개의 Template Angle이 곧 Glass Angle이다.
3. 3점이면 Affine가 Rigid보다 항상 좋다.
4. Local Align이 많을수록 정확하다.
5. Image 좌표끼리 바로 Fit해도 된다.
6. Outlier를 버리면 Align은 성공이다.
7. Align 완료 후 Residual/재촬영은 필요 없다.

10. 면접에서 나올 수 있는 질문

1. **Align과 Calibration 차이?** Calibration은 좌표계 관계, Align은 제품별 Pose.
2. **두 Mark로 X/Y/Theta 가능?** Rigid 전제에서 Pair Angle과 Translation으로 가능.
3. **세 번째 Mark 역할?** Noise 평균화, 오검출/Geometry/Scale 이상 검증.
4. **Template와 Feature Matching 차이?** 전자는 고정 Patch 유사도, 후자는 Keypoint 대응과 Outlier Model Fit.
5. **Score 외 무엇을 검증?** ID, 범위, Pair 거리, Angle, Residual, 반복성.
6. **Global/Local 차이?** 제품 전체 Pose와 특정 영역의 추가 보정.

11. 실습 문제

1. $X \pm 20/Y \pm 15/\theta \pm 1^\circ$ Data를 생성해 고정 ROI와 Dynamic ROI 실패율을 비교한다.
2. Mark Baseline 50/150/300 mm에서 5 μ m 위치 Noise의 각도 Noise를 계산한다.
3. 한 Mark에 0.2 mm Outlier를 넣고 RMS/Max Residual 변화를 측정한다.
4. Template Score와 실제 위치 오차의 Scatter Plot을 만든다.

Phase 9 미니 프로젝트: Alignment Evaluator

Mark Image/Reference Geometry를 입력해 Mark별 Score/좌표, X/Y/Theta, Pair 거리, RMS/Max Residual, ROI Overlay와 실패 사유를 저장한다.

12. Chapter 핵심 요약

- Align은 Reference와 Current 제품의 Pose 차이를 구한다.
- 강체 제품은 최소 자유도의 Rigid Model을 우선한다.
- Match Score와 위치 정확도를 구분한다.
- 3점 이상은 Least Squares와 Residual 검증에 유리하다.
- 모든 Mark는 공통 Calibration 좌표로 변환한다.
- Global Pose 후 필요 영역만 Local Align한다.
- Outlier 제거 사실과 실패 사유를 Result에 남긴다.
- 보정 후 재측정으로 Pivot/Stage Model을 검증한다.

13일차 학습 완료 체크

- ☐ Matching 세 종류의 적용 조건을 설명한다.
- ☐ 2점/N점 Rigid Pose를 계산한다.
- ☐ Score/Residual/Geometry를 함께 판정한다.
- ☐ Global/Local Align을 구분한다.
- ☐ Alignment Evaluator를 설계하거나 구현했다.

다음 학습 예고

14일차에는 Pixel/Image 좌표를 Machine/World 좌표로 바꾸는 Calibration과 Camera Intrinsic/Distortion을 학습한다.

14일차 A. Calibration: Image Coordinate를 Machine Coordinate로 변환하기

Phase 10 · 예상 학습 시간: 12~16시간 · 난이도: 중급→고급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

- ● Calibration의 목적과 Align과의 차이
- ● Pixel/Image/Machine/World Coordinate
- ● Scale, Offset, Rotation, Axis Flip
- ● Similarity/Affine/Homography 선택
- ● Point Correspondence와 Least Squares
- ● Fit/Validation Residual과 Version 관리

실무 목표

Image Point (u, v) 를 Machine Point (X, Y) 로 변환하고 Model의 전제와 단위를 설명한다. FOV 전체의 독립 검증점에서 RMS/Max Error를 평가한다.

2. 선수 지식

- Alignment 심화
- Homogeneous Coordinate와 Matrix 곱
- 1 mm=1000 μ m

TEXT

Calibration: 좌표계 사이의 장기적 관계
Alignment: 매 제품의 현재 Pose 변화

Camera/Lens/WD/Focus/Resolution/ROI Mode가 바뀌면 Calibration 유효성을 재검토한다.

3. 핵심 개념

3.1 왜 필요한가?

Image에서 100 Pixel 이동했다는 사실만으로 실제 거리를 알 수 없다. Scale, 축 방향, Rotation, 원점 Offset과 왜곡이 필요하다.

TEXT

Image (u,v) pixel --Calibration--> Machine (X,Y) mm

3.2 Scale+Offset

FORMULA · CALCULATION

$$X = s_x u + o_x$$

$$Y = s_y v + o_y$$

Image +v와 Machine +Y가 반대면 s_y 가 음수다. 이 Model은 Camera Rotation과 Shear를 설명하지 못한다.

3.3 Affine Transform

FORMULA · CALCULATION

$$\begin{bmatrix} X \\ Y \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

$$[Y] = [a_{21} \ a_{22}][v] \quad [t_y]$$

Translation, Rotation, X/Y Scale, Axis Flip, Shear를 표현한다. 평면 Object의 일반적인 2D Calibration에 사용한다.

3.4 Homography

FORMULA · CALCULATION

$$\begin{bmatrix} X \sim & Y \sim & W \sim \end{bmatrix}^T = H \begin{bmatrix} u & v & 1 \end{bmatrix}^T$$

$$X = X \sim / W \sim, Y = Y \sim / W \sim$$

Plane Perspective를 표현하며 최소 4개 non-collinear Point가 필요하다. Object 높이가 Calibration Plane을 벗어나면 같은 H가 정확하지 않다.

3.5 Model 선택

Model	조건	장점	위험
Scale+Offset	축 평행/균일	단순	Rotation 미반영
Similarity	Rotation+Uniform Scale	강체 관계	X/Y 차이 미반영
Affine	평면/약한 Perspective	실용적	Lens Distortion 미반영
Homography	Plane Perspective	기울기 대응	높이 변화/Overfit
Distortion+Pose	정밀/광각	물리 Model	복잡

최소 자유도로 시작하고 Residual의 공간 Pattern을 확인한다.

3.6 Point Correspondence

TEXT

$(u_i, v_i) \leftrightarrow (X_i, Y_i)$

Point는 중앙과 네 모서리 등 FOV 전체에 고르게 분포한다. 가까운 세 점의 Affine Fit Error가 0이어도 FOV 밖 정확도는 증명되지 않는다.

3.7 Fit과 Validation 분리

FORMULA · CALCULATION

$$e_i = |P_machine_i - Transform(p_image_i)|$$

$$RMS = \sqrt{\sum e_i^2 / N}$$

$$Max = \max(e_i)$$

Fit에 사용하지 않은 Point에서 RMS, Max와 Error Vector Map을 평가한다.

3.8 Stage 기반 Calibration

Stage 이동량과 Image Mark 이동을 대응시킬 수 있다. Stage Backlash/직교도/반복성, Settle, Encoder와 Command 차이, Trigger 시점 Pose를 검증한다.

3.9 Handedness와 Determinant

Image는 아래 +v, Machine은 위 +Y일 수 있다. Affine Linear Part의 Determinant가 음수면 Reflection/Axis Flip이 포함된다. 의도인지 좌표 정의 오류인지 확인한다.

3.10 Version

Calibration ID, Camera/Lens/WD/Focus/ROI Mode, Target ID, Matrix, Fit/Validation RMS·Max와 승인 상태를 Result에 연결한다.

4. 그림으로 이해하기

TEXT

Image (pixel/down +v) → scale/rotation/flip/origin → Machine (mm/up +Y)

TEXT

Fit Points → Matrix → Separate Validation Points → Residual Map → Accept

5. 실제 검사 장비에서 어디에 사용하는가?

- Align Pixel Offset을 Stage 보정량으로 변환
- Defect 좌표를 mm로 저장
- Multi-camera 결과를 공통 좌표로 통합
- Pixel 거리/면적을 실제 단위로 측정
- Robot/Stage가 Image Feature로 이동

6. 숫자로 이해하기

예제 1: 단순 Scale

FORMULA · CALCULATION

$$2 \mu\text{m}/\text{pixel} \times 100 \text{ pixel} = 200 \mu\text{m} = 0.2 \text{ mm}$$

예제 2: Rotation과 Y Flip

$s=0.002 \text{ mm}/\text{pixel}$, 1° Rotation:

FORMULA · CALCULATION

$$A = \begin{bmatrix} s \cos 1^\circ & s \sin 1^\circ \\ s \sin 1^\circ & -s \cos 1^\circ \end{bmatrix}$$

Image Delta $(100, 0)$ 은 Machine Delta $(0.199970, 0.003490) \text{ mm}$ 다.

예제 3: Residual

Error $0.006, 0.008, 0.010, 0.020 \text{ mm}$:

FORMULA · CALCULATION

$$\text{RMS} \approx 0.01225 \text{ mm}, \text{Max} = 0.020 \text{ mm}$$

허용 오차 $\pm 0.015 \text{ mm}$ 라면 RMS만 보면 안 된다.

예제 4: Matrix

FORMULA · CALCULATION

$$X = 0.002u + 0.00002v + 100$$

$$Y = 0.00001u - 0.002v + 250$$

$$(u,v) = (1000, 500) \rightarrow (X,Y) = (102.010, 249.010) \text{ mm}$$

7. C++ 구현

CPP

```
#include <opencv2/opencv.hpp>
#include <algorithm>
#include <cmath>
#include <stdexcept>
#include <vector>

class AffineCalibration final {
public:
    explicit AffineCalibration(const cv::Matx23d& matrix):matrix_(matrix)
    {
        for(double value:matrix_.val)
            if(!std::isfinite(value)) throw std::invalid_argument{"Finite matrix required"};
    }
    [[nodiscard]] cv::Point2d ImageToMachine(const cv::Point2d& p) const noexcept
    {
        return {matrix_(0,0)*p.x+matrix_(0,1)*p.y+matrix_(0,2),
                matrix_(1,0)*p.x+matrix_(1,1)*p.y+matrix_(1,2)};
    }
private: cv::Matx23d matrix_;
};

struct CalibrationQuality final {double rmsMm{},maxMm{};};

[[nodiscard]] CalibrationQuality EvaluateCalibration(
    const AffineCalibration& c,const std::vector<cv::Point2d>& image,
    const std::vector<cv::Point2d>& machine)
{
    if(image.size()!=machine.size()||image.empty())
        throw std::invalid_argument{"Equal non-empty point sets required"};
    double sum=0,maxError=0;
    for(size_t i=0;i<image.size();++i){
        const double e=cv::norm(c.ImageToMachine(image[i])-machine[i]);
        sum+=e*e; maxError=std::max(maxError,e);
    }
    return {std::sqrt(sum/image.size()),maxError};
}
```

`estimateAffine2D/estimateAffinePartial2D` 로 Fit할 때 RANSAC Inlier/Outlier를 기록한다. Fit 성공은 Validation 통과가 아니다.

Unit Test

CPP

```
#include <cassert>
void TestKnownAffine(){
    const AffineCalibration c{{0.002,0.00002,100,0.00001,-0.002,250}};
    const auto p=c.ImageToMachine({1000,500});
    assert(std::abs(p.x-102.010)<1e-9&&std::abs(p.y-249.010)<1e-9);
}
```

8. 실무에서 발생하는 문제

1. 중앙 Point만 사용해 Corner Error 누락.
2. Fit Point로 품질을 평가해 Overfit 통과.
3. Image y Flip 누락으로 Stage 반대 이동.
4. Lens/WD 변경 뒤 Matrix 재사용.
5. Stage Backlash를 Camera 변환으로 Fit.
6. RANSAC Outlier를 숨김.
7. mm/ μ m 혼용으로 1000배 오류.

9. 흔한 오해

1. Calibration은 μ m/pixel 하나다.
2. Align과 Calibration은 같은 Matrix다.
3. 세 Fit Point Error 0이면 정확하다.
4. Homography가 항상 우수하다.
5. RMS가 작으면 모든 위치가 정확하다.
6. Camera가 같으면 Lens 교체 후에도 같다.

10. 면접에서 나올 수 있는 질문

1. **왜 필요한가?** Pixel을 실제 좌표/단위/축으로 연결한다.
2. **Align과 차이?** System 관계와 제품별 Pose의 차이다.
3. **Affine/Homography 차이?** 평행성 유지와 Perspective Plane 표현.
4. **100 Pixel은 몇 μ m?** 위치/방향 Calibration 없이는 모른다.
5. **품질 평가?** 독립 Point의 RMS/Max와 Residual Map.
6. **재Calibration 시점?** 광학/기구/Mode 변경 또는 Drift 초과.

11. 실습 문제

1. Y Flip과 2° Rotation을 포함한 Matrix 작성.
2. 3×3 Grid의 Fit/Validation 분리와 Residual Map 설계.
3. 5 μ m/pixel, 2048 Width에서 0.2% Scale Error의 끝단 오차 계산.
4. 장비 변경별 Calibration 무효화 표 작성.

Phase 10 미니 프로젝트

Point Pair CSV에서 Similarity/Affine을 Fit하고 Matrix, Inlier, Fit/Validation RMS·Max, Residual Overlay와 Version Metadata를 저장한다.

12. Chapter 핵심 요약

- Calibration은 Pixel을 Machine/World 좌표로 연결한다.
- Scale, Origin, Rotation, Flip과 왜곡을 고려한다.
- 최소 Transform Model을 선택한다.
- Fit과 독립 Validation을 분리한다.
- RMS, Max와 공간 Error를 함께 본다.
- 단위와 좌표 방향을 명시한다.

다음 문서

14일차 B. Camera Intrinsic/Extrinsic과 Distortion

14일차 B. Camera Calibration: Intrinsic · Extrinsic · Distortion

Phase 11 · 예상 학습 시간: 10~14시간 · 난이도: 중급→고급 · 중요도: ● 실무에서 중요

1. 이 Chapter에서 배우는 것

- Intrinsic Matrix K : f_x, f_y, c_x, c_y
- Extrinsic: Camera Pose R, t
- Radial/Tangential Distortion
- Target/View 수집과 Reprojection Error
- Undistort와 독립 Validation

실무 목표

OpenCV Camera Calibration 결과의 물리적 의미를 설명하고 왜곡 보정 전후 Residual을 검증한다. Telecentric/작은 FOV 에서 단순 Plane Mapping이 충분한지, 일반 Lens에서 Distortion Model이 필요한지 판단한다.

2. 선수 지식

- Image→Machine Calibration
- Pinhole Camera와 3D→2D 투영의 직관

3. 핵심 개념

3.1 Pinhole Model

FORMULA · CALCULATION

$$s[u \ v \ 1]^T = K[R|t][X \ Y \ Z \ 1]^T$$

$[R|t]$ 는 World를 Camera Coordinate로, K 는 Camera Point를 Pixel로 투영한다.

3.2 Intrinsic

FORMULA · CALCULATION

$$K = \begin{bmatrix} f_x & \text{skew} & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

f_x/f_y 는 Pixel 단위 Focal Length, c_x/c_y 는 Principal Point다. 이상적으로 $f_x \approx f_{\text{mm}}/\text{pixelPitch}_{\text{mm}}$ 지만 실제 값은 Focus와 Model에 따른 유효값이다.

3.3 Extrinsic

Target/World Coordinate를 Camera Coordinate로 바꾸는 Rotation과 Translation이다. View마다 상대 Pose가 달라 $rvec/tvec$ 도 달라진다.

3.4 Radial Distortion

FORMULA · CALCULATION

$$\begin{aligned} x_d &= x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \\ y_d &= y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \end{aligned}$$

중심에서 멀수록 Barrel/Pincushion 위치 Error가 커진다.

3.5 Tangential Distortion

Lens/Sensor 비중심 정렬을 $p1, p2$ 로 모델링한다. 계수를 늘리면 Fit Error는 줄 수 있지만 Data가 부족하면 Overfit한다.

3.6 Target

- Checkerboard
- Symmetric/Asymmetric Circle Grid
- 정밀 Dot/Grid Target

Target Pitch 정확도가 최종 Scale을 제한한다. 종이 출력물은 정밀 Calibration Target으로 부적합할 수 있다.

3.7 좋은 View

- Target이 중앙/모서리를 덮음
- 거리와 Tilt가 다양함
- Blur/Saturation 없음
- 거의 같은 View만 반복하지 않음

장수보다 Parameter를 관측할 Pose 다양성이 중요하다.

3.8 Reprojection Error

FORMULA · CALCULATION

$$e = |p_{\text{detected}} - p_{\text{projected}}|$$

전체 RMS 외에 View별/위치별 Max와 Vector Pattern을 본다. Fit Corner Error가 작아도 Machine 측정 정확도를 보장하지 않는다.

3.9 Undistort

`initUndistortRectifyMap` 으로 Map을 미리 만들고 `remap` 할 수 있다. 보간이 Gray/Edge를 바꾸므로 정말 측정은 Raw Point를 `undistortPoints` 로 보정하는 방식과 비교한다.

3.10 검사 장비 적용 범위

- 일반 Lens/넓은 FOV: 왜곡 Model 중요
- Telecentric/작은 FOV: Grid Residual로 단순 Model 충분성 확인
- 평면 검사: Undistortion+Plane Mapping
- 높이 변화: 단일 Homography 부족 가능

4. 그림으로 이해하기

TEXT

```
World → Extrinsic(R,t) → Camera → Intrinsic(K) → Ideal Pixel
                                     ↓ Distortion
                                     Measured Pixel
```

TEXT

Multiple Views → `calibrateCamera` → Reproject → Error Map
→ Independent Grid Validation → Deploy Map/Version

5. 실제 검사 장비에서 어디에 사용하는가?

- FOV Corner의 치수/위치 Bias 감소
- Multi-camera 통합 전 왜곡 보정
- Mark/Edge의 Plane Mapping
- Lens/Camera 장착 상태 및 Drift 확인

6. 숫자로 이해하기

예제 1: Focal Pixel

FORMULA · CALCULATION

$f=16\text{ mm}$, $\text{pitch}=3.45\text{ }\mu\text{m}=0.00345\text{ mm}$
 $f_x \approx 16/0.00345 \approx 4637.68\text{ pixel}$

예제 2: Radial 항

$k1=-0.1$, $r=0.5$, 다른 항 0:

FORMULA · CALCULATION

$\text{scale}=1-0.1 \times 0.25=0.975$

반경 좌표가 2.5% 안쪽으로 이동하는 단순 예다.

예제 3: Reprojection Error

FORMULA · CALCULATION

$\text{RMS}=0.2\text{ pixel}$, $\text{Sampling}=5\text{ }\mu\text{m/pixel} \rightarrow \text{단순 환산 } 1\text{ }\mu\text{m}$

측정 정확도 $1\text{ }\mu\text{m}$ 를 보장하는 뜻은 아니다.

예제 4: Corner

Corner Error 1.2 Pixel, $4\text{ }\mu\text{m/pixel}$ 이면 단순 위치 Error는 $4.8\text{ }\mu\text{m}$ 다. 전체 RMS로 Corner 문제를 숨기지 않는다.

7. C++ 구현

C++

```
#include <opencv2/opencv.hpp>
#include <algorithm>
#include <cmath>
#include <stdexcept>
#include <vector>

struct CameraCalibrationResult final {
    cv::Mat cameraMatrix, distCoeffs;
    std::vector<cv::Mat> rvecs, tvecs;
    double rmsPixels{}, meanErrorPixels{}, maxErrorPixels{};
};

[[nodiscard]] CameraCalibrationResult CalibrateCamera(
    const std::vector<std::vector<cv::Point3f>>& objectPoints,
    const std::vector<std::vector<cv::Point2f>>& imagePoints,
    const cv::Size imageSize)
{
    if(objectPoints.size()!=imagePoints.size()||objectPoints.size()<3||
        imageSize.width<=0||imageSize.height<=0)
        throw std::invalid_argument{"Invalid observations"};
    cv::Mat K=cv::Mat_<double>(3,3,CV_64F), dist;
    std::vector<cv::Mat> rvecs, tvecs;
    const double rms=cv::calibrateCamera(objectPoints, imagePoints, imageSize,
                                         K, dist, rvecs, tvecs);

    double sum=0, maxError=0; size_t count=0;
    for(size_t i=0; i<objectPoints.size(); ++i){
        std::vector<cv::Point2f> projected;
        cv::projectPoints(objectPoints[i], rvecs[i], tvecs[i], K, dist, projected);
        if(projected.size()!=imagePoints[i].size())
            throw std::runtime_error{"Projection size mismatch"};
        for(size_t j=0; j<projected.size(); ++j){
            const double e=cv::norm(projected[j]-imagePoints[i][j]);
            sum+=e; maxError=std::max(maxError, e); ++count;
        }
    }
    if(count==0) throw std::runtime_error{"No calibration points"};
    return {K, dist, rvecs, tvecs, rms, sum/static_cast<double>(count), maxError};
}
```

검증 Test

1. f_x/f_y 양수, Principal Point 범위
2. View별 RMS/Max와 Corner Vector
3. 독립 Grid의 실제 거리 Error
4. Image Size/ROI/Binning 불일치 거부
5. 보정 전/후 측정 Bias 비교

8. 실무에서 발생하는 문제

1. 중앙에만 Target 배치.
2. 동일 Pose 반복으로 관측성 부족.
3. 부정확한 Target Pitch.
4. Resize/ROI 변경 후 K 재사용.
5. RMS만 저장해 Corner Outlier 누락.
6. 과도한 계수로 Overfit.
7. Remap 보간 Edge Bias.

9. 흔한 오해

1. fx는 Lens 표기 mm와 같은 단위다.
2. Extrinsic은 Camera 고유값이다.
3. RMS 0.2 Pixel이면 측정 정확도도 0.2 Pixel이다.
4. View가 많기만 하면 좋다.
5. Telecentric Lens는 Calibration이 필요 없다.
6. Undistort하면 모든 Error가 사라진다.

10. 면접에서 나올 수 있는 질문

1. **Intrinsic?** fx/fy/cx/cy의 내부 투영 Parameter.
2. **Extrinsic?** World/Target 대비 Camera Pose.
3. **Radial/Tangential?** 반경 Lens 왜곡과 비중심 정렬 왜곡.
4. **좋은 View?** FOV 전체와 다양한 Tilt/거리의 선명한 Target.
5. **평가?** 위치/View별 RMS/Max와 독립 실제 거리.
6. **ROI 변경?** K 조정 근거를 검증하거나 재Calibration.

11. 실습 문제

1. Checkerboard Object Point를 mm로 생성한다.
2. View/Corner별 Reprojection Error Map을 만든다.
3. `undistortPoints` 와 Remapped Edge 위치를 비교한다.
4. Image 0.5배 Resize 시 fx/fy/cx/cy를 계산한다.

Phase 11 미니 프로젝트

Image Set에서 Corner를 검출하고 K/Distortion, View별 RMS/Max, Error Vector, Undistort Preview, 독립 Grid 거리 Error 와 Metadata를 저장한다.

12. Chapter 핵심 요약

- Intrinsic과 Extrinsic을 구분한다.
- Distortion은 위치에 따른 기하 Error다.
- Target 정확도와 View 다양성이 중요하다.
- 전체 RMS 외 위치/View별 Max를 본다.
- 독립 Target의 실제 거리로 검증한다.
- Image Size/ROI Mode는 Calibration 조건이다.

다음 학습 예고

15일차에는 Image/Camera/Stage/Machine/World Coordinate를 하나의 Transform Chain으로 연결한다.

15일차. Coordinate System: Image · Camera · Stage · Machine · Product

Phase 12 · 예상 학습 시간: 10~14시간 · 난이도: 중급→고급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

- 좌표계(Frame) 이름, 원점, 축, 단위
- Image/Camera/Stage/Machine/World/Product Coordinate
- Forward/Inverse Transform과 합성 순서
- Align Pose와 Calibration Transform의 결합
- Stage 이동 시 촬영 Pose 결합
- 좌표 결과의 Traceability와 Unit Safety
- Hand-eye/Camera-to-stage 관계
- 2D Plane Model의 한계

실무 목표

Point 값만 전달하지 않고 어떤 Frame의 어떤 단위인지 계약한다. Transform 이름을 `T_target_source` 로 읽고 여러 Camera/Stage/제품 좌표를 공통 Machine Frame으로 변환한다.

2. 선수 지식

- Image→Machine Calibration
- Align Pose
- 3×3 Homogeneous Transform

FORMULA · CALCULATION

$$p_{\text{target}} = T_{\text{target_source}} \times p_{\text{source}}$$

$T_{\text{machine_image}}$ 는 Image Point를 Machine Point로 바꾸는 Matrix다.

3. 핵심 개념

3.1 좌표는 숫자 두 개가 아니다

(100, 200) 만으로는 의미가 없다.

TEXT

Frame: image-raw / image-undistorted / stage / machine / product

Unit: pixel / μm / mm

Axis: +x/+y direction

Origin: where?

Time/Pose: which frame capture?

3.2 Image Coordinate

보통 좌상단 (0, 0) , 오른쪽 +u, 아래 +v, 단위 pixel이다. Camera ROI/Crop/Binning/Resize가 원점과 Scale을 바꿀 수 있다.

3.3 Camera Coordinate

Pinhole Model에서 Camera Optical Center 원점, 보통 +Z가 광축 방향인 3D 좌표다. Normalized Image Coordinate와 혼동하지 않는다. 2D 평면 검사에서는 Distortion 보정 후 Plane Mapping으로 직접 Machine 좌표를 구할 수도 있다.

3.4 Stage Coordinate

Motion Controller/Encoder가 사용하는 좌표다. Command Position과 Actual Position, Local/Global Frame, 회전 Pivot을 구분한다.

3.5 Machine Coordinate

장비 전체의 공통 Frame이다. 여러 Camera, Stage, Robot, Review 좌표를 통합한다. 장비 기구 Datum과 연결되고 mm 또는 μm 를 사용한다.

3.6 World/Product Coordinate

- World: 공정/Cell/Line 기준 좌표
- Product: 제품 중심/Datum 기준 좌표

Align은 Current Product Frame이 Machine Frame에서 어디에 있는지 구한다.

FORMULA · CALCULATION

$$p_{\text{machine}} = T_{\text{machine_product}} \times p_{\text{product}}$$

$$p_{\text{product}} = \text{inverse}(T_{\text{machine_product}}) \times p_{\text{machine}}$$

3.7 Raw와 Undistorted Image Frame

왜곡 보정 전/후 Pixel Coordinate는 서로 다른 Frame이다. Raw Image에서 검출한 Point에 Undistorted Calibration Matrix를 바로 적용하면 안 된다.

3.8 Transform Chain

FORMULA · CALCULATION

$$p_{\text{machine}} = T_{\text{machine_stage}}$$

$$\quad \times T_{\text{stage_camera}}$$

$$\quad \times T_{\text{camera_image}}$$

$$\quad \times p_{\text{image}}$$

Matrix는 Point에 가까운 오른쪽부터 적용된다. 각 중간 Frame의 Source/Target이 맞지 않으면 합성할 수 없다.

3.9 Fixed Camera와 Moving Camera

Fixed Camera, Moving Product Stage

Camera↔Machine 관계는 고정이고 Product Pose가 Stage와 함께 변한다.

Moving Camera

촬영마다 Stage Actual Pose가 Camera Frame에 들어간다.

FORMULA · CALCULATION

$$p_{\text{machine_i}} = T_{\text{machine_stage}}(\text{capture_i})$$

$$\quad \times T_{\text{stage_camera}}$$

$$\quad \times p_{\text{camera_i}}$$

여러 Mark를 순차 촬영하면 각 Frame의 Timestamp/Stage Pose가 반드시 필요하다.

3.10 Align Pose 방향

Align Module이 반환하는 Pose를 명시한다.

FORMULA · CALCULATION

Forward: $p_{\text{current}} = T_{\text{current_reference}} \times p_{\text{reference}}$

Inverse: $p_{\text{reference}} = \text{inverse}(T_{\text{current_reference}}) \times p_{\text{current}}$

“X=+10”만 전달하지 말고 **reference→current** , Pivot, 단위와 축을 포함한다.

3.11 Physical와 Virtual Correction

- Physical: Stage를 움직여 Current Product Frame을 Reference에 맞춤
- Virtual: Image/ROI/측정 Point를 Product Reference Frame으로 역변환

Stage Correction은 Pivot과 Controller Frame을 포함한 Pose 역변환이다. 단순 부호 반전은 작은 각도의 근사일 수 있다.

3.12 Transform Graph와 단일 경로

좌표계 관계를 Graph로 관리하되 같은 Source→Target에 여러 경로가 생기면 어떤 Calibration Version이 권위 있는지 정한다. Runtime에 임의 Matrix를 섞지 않는다.

3.13 단위 안전

변수명/Type에 단위를 표현한다.

TEXT

```
imagePointPx
machinePointMm
objectScaleUmPerPixel
thetaDegrees / thetaRadians
```

단위 변환은 경계 함수 한 곳에서 수행하고 Result에도 Unit을 저장한다.

4. 그림으로 이해하기

TEXT

```
Raw Image(px)
| distortion correction
Undistorted Image(px)
| plane calibration
Camera/Stage(mm)
| capture stage pose / hand-eye
Machine(mm)
| inverse product align pose
Product(mm)
| line/cell transform
World(mm)
```

FORMULA · CALCULATION

T_{A_B} : B point $\rightarrow$ A point

$T_{A_C} = T_{A_B} \times T_{B_C}$

5. 실제 검사 장비에서 어디에 사용하는가?

- 여러 Mark 순차 촬영 좌표 통합
- Align Result로 Stage 또는 ROI 보정
- NG Defect의 Product/Machine 좌표 저장
- Multi-camera 결과 Fusion
- Review Overlay와 Motion Go-to-position
- Recipe ROI를 Current Image에 배치

6. 숫자로 이해하기

예제 1: 요청 예제의 좌표 계약

가정:

FORMULA · CALCULATION

Current Image Point $p=(1250,850)$ pixel

Reference Pivot $c=(1024,1024)$ pixel

Align Forward Pose: reference $\rightarrow$ current

$tx=+10$ pixel, $ty=-20$ pixel, $\theta=+0.5^\circ$

Current Point를 Product Reference로 되돌린다.

FORMULA · CALCULATION

$p_{ref} = R(-\theta) \times (p-c-t) + c$

$p-c-t=(216,-154)$

$p_{ref} \approx (1238.648, 868.121)$ pixel

예제 2: Reference Pixel $\rightarrow$ Product 실제 좌표

Reference Origin (1024, 1024) , Scale 2 $\mu\text{m}/\text{pixel}$, Product +Y가 Image -v라면:

FORMULA · CALCULATION

$$X = (1238.648 - 1024) \times 2 \approx +429.296 \text{ } \mu\text{m}$$

$$Y = -(868.121 - 1024) \times 2 \approx +311.758 \text{ } \mu\text{m}$$

Offset/Rotation/왜곡이 없는 단순 Scale 예이며 실제 장비는 Calibration Matrix를 사용한다.

예제 3: Transform 합성 순서

Camera Point **(1, 0) mm** , Camera가 Stage에서 +100 mm X Offset, Stage가 Machine에서 20 mm Y에 있다면:

FORMULA · CALCULATION

$$p_{\text{stage}} = (101, 0) \text{ mm}$$

$$p_{\text{machine}} = (101, 20) \text{ mm}$$

예제 4: 단위 오류

0.2 mm 보정량을 **200 μm** 단위 API에 숫자 **0.2** 로 넘기면 1000배 작은 명령이 된다. API 경계에서 명시적으로 변환한다.

7. C++ 구현

C++

```
#include <opencv2/opencv.hpp>
#include <cmath>
#include <stdexcept>

class Transform2d final {
public:
    explicit Transform2d(const cv::Matx33d& m):m_(m)
    {
        for(double v:m_.val) if(!std::isfinite(v))
            throw std::invalid_argument{"Finite transform required"};
        if(std::abs(cv::determinant(cv::Mat(m_)))<1e-12)
            throw std::invalid_argument{"Invertible transform required"};
    }

    [[nodiscard]] static Transform2d FromPose(
        double tx,double ty,double thetaDegrees)
    {
        constexpr double pi=3.14159265358979323846;
        const double r=thetaDegrees*pi/180.0,c=std::cos(r),s=std::sin(r);
        return Transform2d{{c,-s,tx,s,c,ty,0,0,1}};
    }

    [[nodiscard]] cv::Point2d Apply(const cv::Point2d& p) const
    {
        const cv::Vec3d h=m_*cv::Vec3d{p.x,p.y,1};
        if(std::abs(h[2])<1e-12) throw std::runtime_error{"Point at infinity"};
        return {h[0]/h[2],h[1]/h[2]};
    }

    [[nodiscard]] Transform2d Inverse() const {return Transform2d{m_.inv()};}

    // this * rhs means rhs is applied first.
    [[nodiscard]] Transform2d operator*(const Transform2d& rhs) const
    { return Transform2d{m_*rhs.m_}; }
private:
    cv::Matx33d m_;
};
```

Unit Test

C++

```
#include <cassert>
void TestCompositionAndInverse(){
    const auto cameraToStage=Transform2d::FromPose(100,0,0);
    const auto stageToMachine=Transform2d::FromPose(0,20,0);
    const auto cameraToMachine=stageToMachine*cameraToStage;
    const auto p=cameraToMachine.Apply({1,0});
    assert(std::abs(p.x-101)<1e-9&&std::abs(p.y-20)<1e-9);
    const auto back=cameraToMachine.Inverse().Apply(p);
    assert(cv::norm(back-cv::Point2d{1,0})<1e-9);
}
```

제품 코드에서는 `ImagePointPx` , `MachinePointMm` 같은 별도 Type/Wrapper로 잘못된 Frame 합성을 Compile-time 에 줄이는 것이 좋다.

8. 실무에서 발생하는 문제

1. Source/Target Matrix 방향 반대.
2. Degree를 Radian API에 전달.
3. Image +v와 Machine +Y 혼동.
4. Command Stage Pose를 Actual Capture Pose로 사용.
5. Raw Point에 Undistorted Matrix 적용.
6. Camera ROI Origin Offset 누락.
7. Stage Pivot/Local Frame 누락.
8. mm/ μ m 1000배 오류.

9. 흔한 오해

1. 좌표는 x/y만 있으면 된다.
2. Matrix 곱 순서는 바뀌도 된다.
3. Align X/Y는 항상 Image Pixel이다.
4. Inverse는 tx/ty/theta 부호만 바꾸면 된다.
5. Camera가 이동해도 같은 Image→Machine Matrix다.
6. Product와 Machine 원점은 같다.

10. 면접에서 나올 수 있는 질문

1. **Image/Stage/Machine 차이?** Pixel 배열, Motion 축, 장비 공통 Datum.
2. **Transform 이름?** `T_target_source` 는 Source Point를 Target으로 변환.
3. **합성 순서?** Point에 가까운 오른쪽 Matrix부터 적용.
4. **Align Pose 방향?** Reference→Current인지 계약하고 반대는 Inverse.
5. **Moving Camera 주의?** Frame마다 Actual Stage Pose/Timestamp 결합.
6. **단위 오류 방지?** Typed Point/변수명/경계 변환과 Golden Test.

11. 실습 문제

1. 본인 장비의 모든 Frame 원점/축/단위 표를 만든다.
2. Camera→Stage→Machine→Product Matrix Chain을 그린다.
3. ROI Crop Origin (`200,100`) 을 포함한 Raw Pixel 변환을 계산한다.
4. Degree/Radian, mm/ μ m, y Flip 오류를 잡는 Unit Test를 작성한다.

Phase 12 미니 프로젝트: Transform Graph Inspector

Frame과 Versioned Transform을 등록하고 Source→Target 경로, 합성 Matrix, Inverse Round-trip Error, Unit/Axis Metadata와 Point 변환 로그를 보여준다.

12. Chapter 핵심 요약

- Point에는 Frame, Unit, Axis, Origin과 Capture Pose가 필요하다.
- `T_target_source` Naming으로 방향을 고정한다.
- Matrix 합성 순서는 교환되지 않는다.
- Align Forward/Inverse Pose를 구분한다.
- Moving Camera는 Frame별 Stage Actual Pose가 필요하다.
- Raw/Undistorted Image Frame을 구분한다.
- 단위와 좌표 Type을 코드 계약으로 표현한다.

다음 학습 예고

16일차에는 Presence/Position/치수/Scratch/Particle/Crack 검사 알고리즘과 전체 Pipeline을 연결한다.